

Documentation for the IMF datatools

The Datatools Team

2026-07-14

Contents

Abstract	3
1 Introduction	4
2 Installation	5
2.1 Installation on Workstations	5
2.2 Installation on Econometric Support Servers	8
3 Data Resources	12
3.1 Notes on Data and Resources	12
3.2 Country Information	13
3.3 iData	13
3.4 Mapping EcOS to iData	18
3.5 DMXe	22
3.6 SQL	24
3.7 Haver	26
3.8 World Bank	29
3.9 BIS	32
3.10 Troubleshooting	35
4 Further Data Retrieval and Manipulation	36
4.1 Writing to DMXe Files	36
4.2 Data Retrieval Scripts	38
4.3 <code>dataframe_utilities</code>	40
5 Using with other Applications	48
5.1 R	48
5.2 Stata	51
6 Summary	66
6.1 Acknowledgements	66

Abstract

This document summarizes the goals, use cases, and example commands for the `imf_datatools` Python library which can be used to retrieve data, metadata, and check available series for many data resources at the IMF. The tools can be called directly from within Stata and R, and instructions are also given for this.

For the latest info, check out the datatools website at <http://datatools>, where a link to the latest version of this document is also available.

Chapter 1

Introduction

This document describes the `imf_datatools` Python package which can be used to retrieve data from various data sources that are used within the Fund. The goal is to provide an interface that allows users to collect data easily from various resources using the same interface and without consideration for the differences in the underlying data sources. Another goal is to provide users with simple ways to query what kind of series and metadata are available in each database.

Section 2 gives a quick installation guide. Section 3 explains each data resource and associated functions in detail, while Section 4 shows further examples of pulling data from multiple sources simultaneously and also applying calculations (frequency conversion, % change, seasonal adjustment using X13, etc.). Section 5 describes how to use the `imf_datatools` in other languages such as R and Stata.

For Python users, the library can easily be used with

```
# import the library
import imf_datatools
# Get data from iData
df_idata = imf_datatools.get_idata_data('IMF.RES:WEO', 'USA.NGDP.A')

# Get data from DMXe
df_dmxe = imf_datatools.get_dmxe_data(
    r'\\ecnsw12p\ems_shared\pub\datatools\samples\gab.dmx', '646BOP')

# Merge all data
from imf_datatools import dataframe_utilities as idt_utils
df_all = idt_utils.merge_dfs([df_idata, , df_dmxe])

# Print out recent years
print(df_all['2020:'])
```

The library has dependencies on various 3rd-party libraries such as numpy, pandas, requests, BeautifulSoup, but should work with most versions of these libraries as of early 2020. If necessary a requirements.txt file will be created. The `imf_datatools` has been tested but there is no guarantee they work for every case. If any problems are found please contact [Datatools Support](#). The latest information on the tools can be found at <http://datatools>.

Chapter 2

Installation

This section describes how to install the `imf_datatools` on Fund personal machines or in a server environment. To gain access to a server maintained by Econometric Support contact EconometricSupport@imf.org and request access to a server with Python available.

2.1 Installation on Workstations

To use the `imf_datatools` the Python library is necessary, even when using from R or Stata. Python and R users should follow steps in 2.1.1 to install the Python library. R users can then read the instructions on usage in 2.1.2. Stata users can jump to the steps in 2.1.3 which will install both the Python library and the necessary Stata commands.

2.1.1 Python Library Installation

1. Make sure to **install the latest Python from the Software Center** (Version 3.11.11 as of Jun 2026)
2. Open Command Prompt by selecting the Windows button at the bottom left of your screen, and typing “cmd”. When Command Prompt appears as a suggestion, select it.
3. In the command prompt, type or copy and paste the following¹:

```
python \\ecsw12p\ems_shared\pub\datatools\installer.py
```

This should install the latest stable release of the `imf_datatools` Python library. **This command can also be used to update the library to the latest version.**

Once this has been done, the installation can be checked by typing the following in the same command prompt window,

```
python
import imf_datatools
exit()
```

If no error messages appear the installation is complete, and the library is ready to use. If any problems occur, send an email to [Datatools Support](#).

If you would like access to the latest development version of the code to try experimental features, do:

¹This location used to be `\\was.int.imf.org\ecn\ems_shared\pub\datatools\dev\imf_datatools` and while this folder still exists, the new location is recommended.

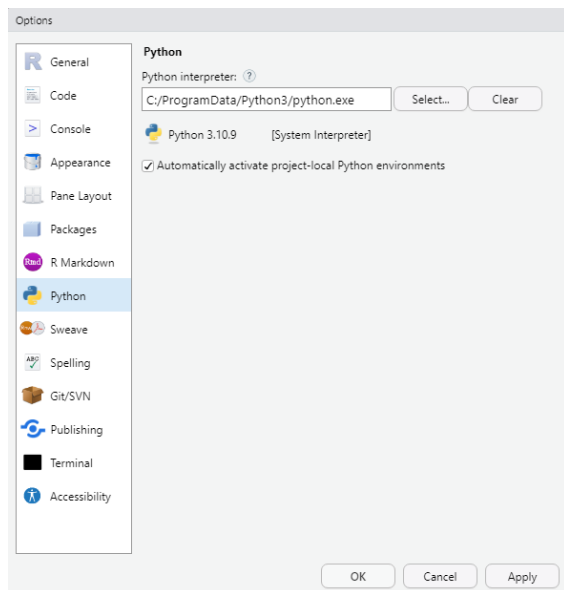
```
python \\ecnsw12p\ems_shared\pub\datatools\installer.py dev
```

2.1.2 Using datatools with R on your workstation

For R users, there are no further installation steps, and below we show how to call the `imf_datatools` from within R.²

After following the steps in 2.1.1 above, **make sure you have the latest R from the Software Center**. Also, **using the `imf_datatools` requires that you have Python installed and have installed the `imf_datatools` library using the above procedure**.

For the latest versions of R it is also necessary to set the Python path. Within RStudio, go to Tools, Global Options, and at the bottom there is a tab for Python.



This should be set to `C:/ProgramData/Python3/python.exe` as shown in the figure above.

To use the `imf_datatools` within R, it is necessary to install the `reticulate` package (this only needs to be done once). In RStudio, execute:

```
install.packages("reticulate")
```

Once the package has been installed, load the library and set the Python version to the correct one:

```
library(reticulate)
# specify the python location.
use_python("C:/ProgramData/Python3", required=TRUE)
```

Import the `imf_datatools` library:

```
imf_datatools <- import("imf_datatools")
```

and it should be ready for use:

```
df <- imf_datatools$get_haver_data("GDP@USECON")
tail(df)
# GDP@USECON
```

²Contact [Econometric Support](#) for general questions on R and how to use the `imf_datatools` in R.

```
# 2019-10-01    21747.4
# 2020-01-01    21561.1
# 2020-04-01    19520.1
# 2020-07-01    21170.3
# 2020-10-01    21494.7
# 2021-01-01    22048.9
```

Diagnosing any problems for R

Below are several checks to do if you have problems using the `imf_datatools` in R.

1. Make sure Python and the `imf_datatools` Python library are correctly installed by opening Command Prompt and doing the following:

```
ipython # start ipython session
import imf_datatools
imf_datatools?
Type:      module
String form: <module 'imf_datatools' from 'C:\\ProgramData\\Python3\\lib\\site-packages\\imf_datatools\\
File:      c:\\programdata\\Python3\\lib\\site-packages\\imf_datatools\\__init__.py
Docstring:  <no docstring>
```

The above should confirm that the `imf_datatools` library is installed and in the correct location. If not, please refer to the installation instructions above.

2. Confirm that the Python path is set as above.
3. Check your Python configuration with

```
py_config()
python:      C:/ProgramData/Python3/python.exe
libpython:   C:/ProgramData/Python3/python310.dll
pythonhome:  C:/ProgramData/Python3
version:     3.10.9 (tags/v3.10.9:1dd9be6, Dec  6 2022, 20:01:21) [MSC v.1934 64 bit (AMD64)]
Architecture: 64bit
numpy:       C:/ProgramData/Python3/Lib/site-packages/numpy
numpy_version: 1.24.4
```

NOTE: Python version was forced by RETICULATE_PYTHON

and if you have mistakenly installed a different version of Python, please delete it and > make sure that the Python path is set correctly.

4. If you have imported other R libraries such as `rio` which also have an `import` function, this may mask the `import` function that is part of the `reticulate` library, resulting in an error like

```
import("imf_datatools")
Error in import("imf_datatools") : No such file
```

In this case you can specify that you want to use the `import` function from the `reticulate` library by specifying `reticulate::import("imf_datatools")` instead of `import("imf_datatools")`. You can also load the `reticulate` library **after loading all other libraries** (in which case the `import` function from all other libraries are masked).

5. Also see Sec.5.1 as for the latest R, the application of timezones may affect the dates displayed.

2.1.3 Stata Installation Instructions for your Workstation

For Stata users it is necessary to install the additional Stata commands that call the `imf_datatools`. **First, make sure you have Stata version 16 or higher, preferably the latest one available in the Software Center.** Also make sure you install the latest Python from the Software Center as well.

2.1.3.1 Setting the Python Path in Stata

If you are using Python for the first time in Stata, or if you have updated Python, you need to update Stata's Python path.

To do this, within Stata, execute

```
set python_exec C:\ProgramData\Python3\python.exe, perm
```

and this will set Python to your latest version. This needs to be done only once.

2.1.3.2 Running `installdatatools.ado` to install the datatools libraries

To use the `installdatatools` Stata command, copy the file `installdatatools.ado` under `\\ecnsw12p\ems_shared\pub\data` to your computer's `C:\ado\plus\i` folder. Next, issue the following command in Stata to install all necessary libraries for the datatools:

```
installdatatools
```

to install both Python and Stata files for datatools. Several black windows may appear and it may take a few minutes to complete. Please do not try to issue any further Stata commands during this time.

If you get the error message

You must install the latest Python from Software Center. Please install Python first and then install datatools

this means there is an old copy of the file `installdatatool.ado` in Stata's path. Run the command `sysdir` to check Stata's path and remove any older versions of `installdatatools.ado`.

If there are any problems please contact Datatools-Support@imf.org.

If you would like access to the latest development version of the code to try experimental features, do:

```
installerdatatools dev
```

2.1.3.3 Checking installation

Once installation is complete, type the following in Stata to see the help files (all examples are clickable):

```
help idatause
help dmxeuse
```

Try running the following command in Stata to test if it works:

```
idatause databases, clear
```

2.2 Installation on Econometric Support Servers

The servers maintained by Econometric Support provide a better computing environment than personal machines issued by the Fund, with more CPU and RAM available. Again, the Python library needs to be installed even if using the

`imf_datatools` from R or Stata. 2.2.1 shows the installation instructions for Python, followed by instructions for R (2.2.2) and Stata (2.2.3).

2.2.1 Installing the Python Library on a Server

To install/use the `imf_datatools` Python library on the servers it is easiest to use a server which already has an environment set up. Send an email to [Econometric Support](#) requesting access to a server containing the `imf_datatools`. Access rights as well as details of how to use the environment will be provided.

Pre-built environments that are mostly identical to what is/was available in the Software Center are available, for Python 3.6.8, 3.8.10 and 3.10.9. This allows users to execute code that was written for these environments without having to make small changes.

To test the environment, do the following in Command Prompt:

```
python
import imf_datatools
exit()
```

If no error messages appear the installation is complete, and the library is ready to use.

If the datatools are not available on your server or if you want to install them in your own virtual environment, copy the folder `\\ecnsw02d\ems_shared\pub\datatools\stable\v1.1.6\imf_datatools\imf_datatools` to under your virtual environment's `lib\site-packages` folder.

2.2.2 Using datatools with R on econometric support server

To use the `imf_datatools` in R on any econometric server, the following steps are needed to correctly set the environment:

1. Make sure that your R location in RStudio is specified using a mapped drive and NOT a network path.

For example, if your R version in Tool -> Global Options... -> General is `\\ecnsw02d\app\r\R-4.4.0`, you need to map `\\ecnsw02d\apps` from the server you are on and set your R location using the mapped drive.

- To map the drive:
 - In your file explorer, choose “This PC” from the left side of the window
 - Click the “Computer” tab then “Map Network Drive” button
 - Choose a drive letter like V and enter `\\ecnsw02d\apps` in the folder field then press finish.
 - A drive with the letter you chose now shows in your “This PC”
- How to set your R path.
 - In RStudio, go to Tools -> Global Option... -> General and click `change...` next to the R Version box.
 - Click browse in the dialog box that pops up and type or navigate to your R location using the newly created mapped drive location as follows: `"V:\r\R-4.4.0\bin\x64"`

NOTE: A previous version of this documentation had `\\ecnsw02d\app\r\r_latest` as the R location. The version of the `reticulate` R library in this folder is incompatible with versions of Python above 3.10. It is now recommended to use the version-numbered R installations like `\\ecnsw02d\app\r\R-4.4.0` and not use `\\ecnsw02d\app\r\r_latest` or `\\ecnsw02d\app\r\r_stable`.

2. Check `HTTPS_PROXY` and `HTTP_PROXY`, using `Sys.getenv()` and make sure they are NOT set.

```
Sys.getenv("HTTPS_PROXY")
Sys.getenv("HTTP_PROXY")
```

If the outputs are not empty, use `Sys.setenv()` to set them to empty.

```
Sys.setenv("HTTPS_PROXY=""")
Sys.setenv("HTTP_PROXY=""")
```

Once the steps above are completed, run the following commands to import your `imf_datatools` into your R environment.

```
library(reticulate)
imf_datatools <- import("imf_datatools")
```

For help, contact [Econometric Support](#) requesting access to a server with the `imf_datatools` installed and instructions on how to use in R.

2.2.3 Stata Installation Instructions for Server Users

First, follow the steps above in 2.2.1 using the Simple method (recommended) or the Advanced method to install the `imf_datatools` Python library.

Set Python Within Stata

In the below examples, the Python environment with the `imf_datatools` library installed is given as `D:\Apps\sss_env`, modify as necessary. Issue the following two commands so that Stata points to the correct version of Python:

```
python set exec "D:\Apps\sss_env\sss_env\python.exe", permanently
python set userpath "D:\Apps\sss_env\sss_env\Lib\site-packages", permanently
```

These commands need to be executed just once, and must be done regardless of whether you are using the Simple method or Advanced method.

Install the Stata Command Files

This applies only if you are using the Advanced method to create your own environment. Copy the file `\\ecnsw12p\ems_shared\pub\datatools\stable\imfdatause\py\imf_stata_utils.py` to under `D:\data\[username]\_sss_e` (change your path based on where your env is).

Create the folder `D:\data\[username]\ado` for Stata files. Within Stata, issue the following commands:

```
cd D:\data\[username]\ado
copy "\\ecnsw12p\ems_shared\pub\datatools\stable\imfdatause\idatause.ado" ., replace
copy "\\ecnsw12p\ems_shared\pub\datatools\stable\imfdatause\idatause.sthlp" ., replace
copy "\\ecnsw12p\ems_shared\pub\datatools\stable\imfdatause\dmxeuse.ado" ., replace
copy "\\ecnsw12p\ems_shared\pub\datatools\stable\imfdatause\dmxeuse.sthlp" ., replace
copy "\\ecnsw12p\ems_shared\pub\datatools\stable\imfdatause\sqluse.ado" ., replace
copy "\\ecnsw12p\ems_shared\pub\datatools\stable\imfdatause\sqluse.sthlp" ., replace
```

This will copy all Stata command files to the `D:\data\[username]\ado` folder.

Finally, for each session of Stata on the server, if you are using the Advanced method, the following command must be executed:

```
adopath + D:\data\[username]\ado
```

This adds the folder `D:\data\[username]\ado` to Stata's path so that the command files are found, and this command can be put in Stata `.do` files so it is automatically executed.

To test that installation is complete, try

```
python:import imf_datatools
```

and if no errors are shown you can now execute commands like

```
idatause databases, clear
```

Chapter 3

Data Resources

This section goes over the available data sources and the associated functions for each resource. **In principle the `imf_datatools` provides the data and metadata as is from the original resource, without modification.** This leads to an inconsistent interface for each resource but instead of trying to modify each resource into a uniform interface we provide users with what is natively available.

In all cases the main output of data retrieval will be a `pandas.DataFrame`, the standard in Python when dealing with tabular data. As the API for the `pandas` library tends to change over time, there may be some dependencies on the version of `pandas` used, but for the most part the libraries are written so that they do not use exotic parts of the library and the interface should be rather stable. The current version (as of 2020) is written with version 0.24.2.

Many functions have an option `debug` which is set to `False` by default, but can be set to `True` so that more information is printed to screen, and in some cases some results are written out to the local folder where the command was executed.

3.1 Notes on Data and Resources

For all retrieved time series data, the data will have a `pd.DatetimeIndex` at the beginning of the period. For example, monthly data for February 2020 will have a date of 2020-02-01, quarterly data for 20Q2 will have a date of 2020-04-01, and annual data for 2019 will have a date of 2019-01-01. The reason for this is because it allows combining data with different frequencies. Data from Haver allows the option of aligning this date to be the end of period, and if necessary this can be implemented in the future for other resources.

The `imf_datatools` can be expanded to support other data resources. For example, there are many public APIs as well as commercial products/services available. However, to add a data resource, the following considerations are necessary.

1. Accessibility

Some web services/APIs require subscriptions and/or API keys which will be necessary for rapid data retrieval. Any code written would have to read in each user's API key. While this is possible, it leads to some setup on the user side.

2. Licensing

Some licenses do not allow for indiscriminate distribution of data even among Fund users, so allowing access to data that is then distributed can lead to legal problems.

Some examples of frequently used resources are CEIC ¹, World Integrated Trade Solution (WITS) ², BEA ³, BLS ⁴,

¹<https://info.ceicdata.com/api-and-data-feed-solution>

²<https://wits.worldbank.org/witsapiintro.aspx?lang=en>

³<https://apps.bea.gov/API/signup/index.cfm>

⁴<https://www.bls.gov/developers/>

Federal Reserve ⁵, Euro Stat ⁶, ECB ⁷, UN Comtrade ⁸. Many central banks also host various data services. There are also data aggregation services such as Quandl <https://www.quandl.com/>, dbnomics <https://db.nomics.world/> which pull data from various resources and make them available.

If there are suggestions for further data resources contact [Econometric Support](#) to see if that resource can be added.

3.2 Country Information

Several functions exist to retrieve information on countries and their groups. In previous versions of the `datatools` these functions were under the library for EcOS data retrieval. Starting in Jun 2026 these functions are now directly under the `imf_datatools` so to call do

```
import imf_datatools

groups = imf_datatools.get_weo_country_groups()

countryinfo = imf_datatools.get_weo_country_info()

ebv = imf_datatools.get_ebv_country_info()
```

For Stata these functions are put under `idatause`, see Sec. 5.2.

`get_weo_country_groups(save=False)`

The RES WEO team maintains an Excel file containing information on country groups and country codes [here](#). This function returns

This function returns a `pandas.DataFrame` containing the contents of the sheet 2. **Country Groups**. The column `groupcode` which contains codes like G001 for **World** is set to be the index.

This function will be useful for getting further information on what codes like G001 mean.

`get_weo_country_info(save=False)`

Similar to `get_weo_country_groups()`, this function returns a `pandas.DataFrame` containing information on WEO countries from the Excel file that the RES WEO team produces. In this case it returns the contents of the sheet 5. **Group Dummies (iData)**, which contains the 3-letter ISO code, country name, and a value of 1 if the country belongs to various country groups defined within the groups defined in the output of `'get_weo_country_groups()'`.

`get_ebv_country_info(save=False)`

This function utilizes the [EBV](#) (Enterprise Business Vocabularies) to return a combined table of country information merged with information from the World Bank. The original csv file is available from this [page](#) under the link [Excel export](#) (it is csv).

Contact the [EBV team](#) for information on the EBV or contents of the data.

3.3 iData

The iData system is the new (as of 2025) Fund-wide data system that has replaced many of the previous data resources including EcOS, EDI, the external-facing data.imf.org, and Data Mapper. These systems have now been decommissioned.

⁵<https://research.stlouisfed.org/docs/api/fred/>

⁶<https://ec.europa.eu/eurostat/web/json-and-unicode-web-services>

⁷<https://sdw-wsrest.ecb.europa.eu/help/>

⁸<https://comtrade.un.org/data/doc/api/>

```
get_databases(keyword=None, searchmode='or', refresh=False, debug=False)
```

Returns a `pandas.DataFrame` containing the available datasets. Once the set of datasets is obtained this is stored in memory so additional calls will not refresh this unless the optional variable `refresh` is set to `True`.

The variable `keyword` is a `str` or iterable of `strs` that allows filtering on dataset names and descriptions (all filters are case-independent). The default `searchmode` is `or` so that for example `keyword=['sta', 'res']` will return any database that contains either “sta” or “res” in its name or description. Setting `searchmode="and"` will provide only databases that contain all specified keywords. Setting `keyword='sta'` will return any database that has the letters “sta” in them.

```
from imf_datatools import idata_utilities

# Get all databases available to the user
datasets = idata_utilities.get_databases()
```

```
get_dimensions(db:str, keyword=None, searchmode='or', refresh=False, debug=False)
```

To query data from iData you must specify the dataset name (the index of the datasets from `get_databases()`), and a query keyword that combines the dimension values of each dimension of the dataset. For example the `IMF.STA:CPI` database has five dimensions `COUNTRY`, `INDEX`, `COICOP_1999`, `TYPE_OF_TRANSFORMATION` and `FREQUENCY`. To retrieve data you must first know these dimensions, then filter on valid values along each dimension. The function `get_dimensions()` provides these dimensions and their mnemonics, and the following `get_dimension_values()` provides the valid candidates for each dimension.

For both `get_dimensions()` and `get_dimension_values()`, within a single session the values are stored in-memory so they are not retrieved multiple times. To force a refresh use the `refresh=True` option.

For example for the `IMF.STA:CPI` dataset,

```
from imf_datatools import idata_utilities

db = 'IMF.STA:CPI'

dims = idata_utilities.get_dimensions(db)
```

gives

	Description	Dimension Order
<code>COUNTRY</code>	Country	0
<code>INDEX_TYPE</code>	Index type	1
<code>COICOP_1999</code>	Expenditure Category	2
<code>TYPE_OF_TRANSFORMATION</code>	Type of Transformation	3
<code>FREQUENCY</code>	Frequency	4

Similar to `get_databases()`, the optional variables `keyword` and `searchmode` allow filtering on the dimension mnemonic and `Description` column. Again the default `searchmode` is “or” so that specifying `keyword=['country', 'type']` would match the dimensions `COUNTRY`, `INDEX_TYPE`, `TYPE_OF_TRANSFORMATION` above.

```
get_dimension_values(db, dimension, keyword=None, searchmode='or', refresh=False, debug=False)
```

This function provides the available candidates for a given dimension for a db. The results are stored in-memory and after initial retrieval use these results unless `refresh=True` is given.

The optional variables `keyword` and `searchmode` are the same for `get_databases()` and `get_dimensions()`, they allow filtering on the dimension value mnemonics or descriptions.

```
from imf_datatools import idata_utilities

db = 'IMF.STA:CPI'
```

```
countrylist = idata_utilities.get_dimension_values(db, 'COUNTRY')
```

gives

Code	Name
AFG	Afghanistan, Islamic Republic of
ALB	Albania
DZA	Algeria
AGO	Angola
AIA	Anguilla, United Kingdom-British Overseas Terr...
...	...
WBG	West Bank and Gaza
YEM	Yemen, Republic of
ZMB	Zambia
ZWE	Zimbabwe
G163	Euro Area (EA)

[201 rows x 1 columns]

Filtering on the results:

```
db = 'IMF.STA:CPI'
```

```
# Filter on "republic"
```

```
republics = idata_utilities.get_dimension_values(db, 'COUNTRY', keyword='republic')
```

```
# Show the first 5
```

```
republics[:5]
```

Code	Name
AFG	Afghanistan, Islamic Republic of
ARM	Armenia, Republic of
AZE	Azerbaijan, Republic of
BLR	Belarus, Republic of
CAF	Central African Republic

```
# Filter on "republic" or "kingdom"
```

```
republic_kingdom = idata_utilities.get_dimension_values(db, 'COUNTRY', keyword=['republic', 'kingdom'])
```

```
# Check how many "republic" or "kingdom"s there are
```

```
len(idata_utilities.get_dimension_values(db, 'COUNTRY', keyword=['republic', 'democratic'], searchmode='or'))
```

```
47
```

```
# Check how many "republic" AND "kingdom"s there are
```

```
len(idata_utilities.get_dimension_values(db, 'COUNTRY', keyword=['republic', 'democratic'], searchmode='and'))
```

```
5
```

```
get_idata_data(db:str, key, start=None, end=None, params=None, longformat=False, panel=None, debug=False)
```

To retrieve data from iData, specify the dataset name `db` from `get_databases()`, then specify the query key which is a `str` concatenating the dimension values of each dimension separated by a “.”. To know the dimensions and their valid values use the functions `get_dimension(db)` and `get_dimension_values(db, dim)` respectively.

1683	USA	CPI	_T	IX	M	2024-12-01	144.736088
1684	USA	CPI	_T	IX	M	2025-01-01	145.683553
1685	USA	CPI	_T	IX	M	2025-02-01	146.330636
1686	USA	CPI	_T	IX	M	2025-03-01	146.659451
1687	USA	CPI	_T	IX	M	2025-04-01	147.116216

[1688 rows x 7 columns]

Finally, setting the option `panel` to a valid dimension name like `COUNTRY` will set the data in panel format, where the data is organized by the specified dimension and dates with one column grouping together the remaining dimensions:

```
key = 'USA+JPN.CPI._T..M'
df = idata_utilities.get_idata_data(db, key=key, panel='COUNTRY')
```

	COUNTRY	dates	CPI._T.IX.M	CPI._T.POP_PCH_PA_PT.M	CPI._T.YOY_PCH_PA_PT.M
0	JPN	1955-01-01	16.771486	NaN	NaN
1	JPN	1955-02-01	16.771486	0.000000	NaN
2	JPN	1955-03-01	16.663512	-0.643794	NaN
3	JPN	1955-04-01	16.807480	0.863973	NaN
4	JPN	1955-05-01	16.645524	-0.963593	NaN
...	...	...	...	...	...
1683	USA	2024-12-01	144.736088	0.035500	2.888057
1684	USA	2025-01-01	145.683553	0.654616	3.000483
1685	USA	2025-02-01	146.330636	0.444170	2.821549
1686	USA	2025-03-01	146.659451	0.224707	2.390725
1687	USA	2025-04-01	147.116216	0.311446	2.311289

[1688 rows x 5 columns]

The option `panel` is mutually exclusive with `longformat=True`.

```
get_idata_metadata(db:str, key, debug=False)
```

While each dimension and its associated values' descriptions can be retrieved using `get_dimensions()` and `get_dimension_values()`, there may be other optional metadata associated with the data. The `get_idata_metadata()` function captures all of the available metadata, which includes for example

- scale
- date updated
- latest period for data before projections
- unit
- methodology

To retrieve this information, the database name `db` must be specified as a valid database name from the index of `get_databases()`, e.g., `IMF.RES:WEO`, `IMF.STA:CPI`, etc. The `key` is very similar to what is used in `get_idata_data()` and can be a `str` representing the dimension values of interest separated by a period. In cases where a dimension value is left open, e.g., `.NGDP.` for `IMF.RES:WEO`, values will be retrieved for all open dimensions. In the case of `.NGDP.`, all countries and frequencies will be returned. Like in `get_idata_data()`, if multiple values from the same dimension are wanted, specify them with a `+` in between, e.g., `.NGDP+LUR..`

The function will return a `pandas.DataFrame` containing the metadata for available series in each row. In cases where there is no matching metadata, `None` is returned. In most cases this function utilizes metadata is put in for databases like `WEO`, `GEE`, `GAS` which `RES` produces, and the metadata will be associated with each individual series.

```
# Get metadata from WEO
db = 'IMF.RES:WEO'
# Get metadata for indicator NGDP
```

```
# and leave other dimensions blank
```

```
key = '.NGDP.'
```

```
meta = idata_utilities.get_metaidata_data(db, key)
```

	indicator	country	primary_domestic_currency	methodology	
1	NGDP	NZL	New Zealand dollar	System of National Accounts (SNA) 2008	Starting with the
2	NGDP	FRA	Euro	European System of Accounts (ESA) 2010	All values are se
3	NGDP	PNG	Papua New Guinea kina	System of National Accounts (SNA) 2008	Historical data s
4	NGDP	NAM	Namibian dollar	System of National Accounts (SNA) 1993	
..	...	...	...	...	
192	NGDP	PLW	US dollar	System of National Accounts (SNA) 1993	Fiscal year data
193	NGDP	MWI	Malawian kwacha	System of National Accounts (SNA) 2008	Since 2011 we hav
194	NGDP	SRB	Serbian dinar	European System of Accounts (ESA) 2010	The country in it
195	NGDP	KAZ	Kazakhstani tenge	System of National Accounts (SNA) 1993	Data prior to 199
196	NGDP	SAU	Saudi riyal	System of National Accounts (SNA) 2008	Historical data s

```
[197 rows x 22 columns]
```

```
# Get metadata from CPI
```

```
db = 'IMF.STA:CPI'
```

```
# Get metadata for index type of CPI, COICOP of _T (All Items)
```

```
# and leave other dimensions blank
```

```
key = '.CPI._T.'
```

```
meta = idata_utilities.get_metaidata_data(db, key)
```

	JPN.CPI._T.IX.M	JPN.CPI._T.POP_PC	index_type	coicop_1999	type_of_transformation	
0	CPI	_T	IX	CPI,inflation,consumer prices,COICOP,price ind...		ABW
1	CPI	_T	IX	CPI,inflation,consumer prices,COICOP,price ind...		AFG
2	CPI	_T	IX	CPI,inflation,consumer prices,COICOP,price ind...		AGO
3	CPI	_T	IX	CPI,inflation,consumer prices,COICOP,price ind...		ALB
4	CPI	_T	IX	CPI,inflation,consumer prices,COICOP,price ind...		ARE
..	...	...	...	...		...
187	CPI	_T	IX	CPI,inflation,consumer prices,COICOP,price ind...		WSM
188	CPI	_T	IX	CPI,inflation,consumer prices,COICOP,price ind...		YEM
189	CPI	_T	IX	CPI,inflation,consumer prices,COICOP,price ind...		ZAF
190	CPI	_T	IX	CPI,inflation,consumer prices,COICOP,price ind...		ZMB
191	CPI	_T	IX	CPI,inflation,consumer prices,COICOP,price ind...		ZWE

```
[192 rows x 12 columns]
```

Note that as seen in the above examples, the attributes returned for each dataset will be different. The WEO dataset contains columns such as `primary_domestic_currency`, `methodology`, `start_end_months_of_reporting_year`, `base_year`, `latest_actual_annual_data`, `unit`, `country_update_date`, which can be used both for analysis as well as metadata associated with the data.

3.4 Mapping EcOS to iData

For the transition from EcOS to iData, there have been several changes:

- The country codes used are now ISO3 instead of IMF codes like “111” for United States.
- The series codes have changed and in many cases have been split from one series code across multiple dimensions.
- Some databases like IFS have been split up into multiple databases.

- Some databases and series codes have been discontinued.

To assist this transition, there is a new module `idata_mapper.py` which translates the above. It should be noted that this is only usable for “standard” EcOS databases that have as inputs country, series code, frequency and optionally a counterpart country. As a special case Bloomberg tickers can also be translated. For databases like those used in `ecos_sdmx_utilities.get_ecos_commodity_data()` that do not have a country, this will not work.

Basic usage examples are below. Note that as EcOS has already been decommissioned, in the below examples commands to get any information from EcOS no longer work.

Map EcOS database name and series code to iData equivalents. EcOS database name should be same as what is used to query EcOS data, e.g., “WEO_WEO_PUBLISHED”.

```
from imf_datatools.idata_mapper import *
from imf_datatools import ecos_sdmx_utilities
from imf_datatools import idata_utilities

ecos_db = 'WEO_WEO_PUBLISHED'
ecos_series = 'NGDP'
idata_db, idata_series = map_db_and_series(ecos_db, ecos_series)
# 'IMF.RES.WEO:WEO_LIVE_2025_APR_VINTAGE', 'NGDP'
```

Get full mapping data available for a given EcOS database using option `ecos_series="ALL"`:

```
ecos_db = 'WEO_WEO_PUBLISHED'
mapping = get_mapping_data(ecos_db, ecos_series='all')
```

	EcosSeriesCode	IDataSeriesCode	DatabaseId	DatabaseRedirect
Id				
38005398	911BG_BP6	ARM.BG_BP6.*	21	IMF.RES.WEO:WEO_LIVE
38005399	911GB_PR.A	ARM.GB_PR.A	21	IMF.RES.WEO:WEO_LIVE
38005400	911GB_PR.Q	ARM.GB_PR.Q	21	IMF.RES.WEO:WEO_LIVE
38005401	911GB_PR	ARM.GB_PR.*	21	IMF.RES.WEO:WEO_LIVE
38005402	911GGXCNL_GDP_M.A	ARM.GGXCNL_GDP_M.A	21	IMF.RES.WEO:WEO_LIVE
...	...	...	...	...
38035573	316NPGDPX0.A	BRB.NPGDPX0.A	21	IMF.RES.WEO:WEO_LIVE
38035574	316NPGDPX0.Q	BRB.NPGDPX0.Q	21	IMF.RES.WEO:WEO_LIVE
38035575	316NPGDPX0	BRB.NPGDPX0.*	21	IMF.RES.WEO:WEO_LIVE
38035576	316NC_USD_WGT.A	BRB.NC_USD_WGT.A	21	IMF.RES.WEO:WEO_LIVE
38035577	316NC_USD_WGT.Q	BRB.NC_USD_WGT.Q	21	IMF.RES.WEO:WEO_LIVE

[414000 rows x 4 columns]

Map IFS, which is split into multiple databases:

```
ecos_db = 'ECDATA_IFS_Latest_Published'
ecos_series = 'PCPI_IX'
idata_db, idata_series = map_db_and_series(ecos_db, ecos_series)
# 'IMF.STA:CPI', 'CPI.T.SPR_IX'
```

Map Bloomberg ticker (fields are same as in EcOS):

```
ecos_ticker = 'VIX Index'
field = 'PX_LAST'
idata_db, idata_ticker = map_bloomberg_ticker(ecos_ticker.upper())
# 'IMF.CSF:BBGDL', 'VIX_INDEX'
```

Translate country from EcOS IMF Codes to ISO3. Note that country groups depend on database, so if there are any

special non-ISO3 country groups, it is necessary to specify the database, default is to use WEO.

```
ecos_country = '111' # IMF code
idata_country = translate_ecos_country(ecos_country)
# 'USA'
```

Compare data from EcOS and iData by translating EcOS country, series code into iData equivalents:

```
ecos_db = 'ECDATA_CPI_LATEST_PUBLISHED'
ecos_series = 'PCPI_IX'
ecos_country = '111'
freq = 'Q'

# Get EcOS data
# This no longer works as EcOS is decommissioned
df_ecos = ecos_sdmx_utilities.get_ecos_sdmx_data(ecos_db, ecos_country, ecos_series, freq=freq)
df_ecos.columns = [c + '_ecos' for c in df_ecos.columns]

# Map to iData db and series
idata_db, idata_series = map_db_and_series(ecos_db, ecos_series)
idata_country = translate_ecos_country(ecos_country)
# Get iData data
from imf_datatools import idata_utilities
key = idata_country + '.' + idata_series + '.' + freq
df_idata = idata_utilities.get_idata_data(idata_db, key)
# Not recommended
# df_idata = get_idata_data_using_ecos(ecos_db, ecos_country, ecos_series, freq, ecos_counterpart=None)
df_idata.columns = [c + '_idata' for c in df_idata.columns]

# Merge
df = df_ecos.merge(df_idata, left_index=True, right_index=True, how='outer')

      111PCPI_IX.Q_ecos  USA.CPI._T.IX.Q_idata
dates
1955-01-01      12.244589      12.244589
1955-04-01      12.244589      12.244589
1955-07-01      12.305736      12.305736
1955-10-01      12.321022      12.321022
1956-01-01      12.290449      12.290449
...
2024-04-01      143.968241      143.968241
2024-07-01      144.403145      144.403145
2024-10-01      144.727986      144.727986
2025-01-01      146.224547      146.224547
2025-04-01      147.488598      147.488598

[282 rows x 2 columns]
```

Below each function is explained in detail.

```
get_mapping_data(ecos_db:str, ecos_series:str, force_update=False, debug=False)
```

This is mostly a utility function to see the internal working of mapping between EcOS and iData. Most users should directly use `map_db_and_series`.

Get mapping between EcOS database and series code to iData equivalents. Specify `ecos_db` and `ecos_series`,

these should be the same as inputs into `ecos_sdmx_utilities.get_ecos_sdmx_data()`. For example, `ecos_db = 'WEO_WEO_PUBLISHED'` and `ecos_series = 'NGDP'`. Returns a `pandas.DataFrame` containing the mapping of EcOS series codes in the given EcOS database for the specified `ecos_series`.

The argument `ecos_series = 'ALL'` is a special case where all series for `ecos_db` will be mapped, this may take time as some of the mapping databases are very large. If possible, the output will be written out to the folder `idata_mapper/` in the current working directory.

`get_mapping(ecos_db:str, ecos_series:str, force_update=False, debug=False)`

This is mostly a utility function to see the internal working of mapping between EcOS and iData. Most users should directly use `map_db_and_series`.

Returns a `str` `idata_db`, dict of `series_mapping`, and dict of dict `dict_series_idata_db`, where `idata_db` is the corresponding iData database name, `series_mapping` is dict between the EcOS series code `ecos_series` and the corresponding iData series code, and `dict_series_idata_db` is a mapping of each series code in `ecos_db` to the iData database it has been mapped to. This only applies to databases like IFS which have been split into multiple databases in iData, and if the database has not been split, this will be `None`.

The results are cached so that the first time the function is run within a session, the results are retrieved from a SQL database and stored internally in memory. The second time the function is run on the same code the results are retrieved from this memory. Using option `force_update` will update results.

In the case of any exceptions, a triplet of (`None`, `None`, `None`) is returned.

`map_db_and_series(ecos_db, ecos_series)`

Map a given `ecos_db` and `ecos_series` to their iData equivalent. Returns a tuple of `strs` `idata_db` and `idata_series`, and if something goes wrong returns `None`, `None`. In cases where the original EcOS database has been split into multiple databases will correctly identify the new iData database.

`map_bloomberg_ticker(ecos_ticker)`

Equivalent of `map_db_and_series` for `ecos_db = 'ECDATA_BLOOMBERG'`, that is, maps a Bloomberg ticker in EcOS to the equivalent in iData. In most cases this is simply replacing any spaces in the EcOS version into underscores for the iData version.

Note that tickers for the input in the the EcOS function `'get_ecos_sdmx_bloomberg()'` are case-sensitive, that is only works for example with “VIX Index” and not “VIX INDEX”, whereas it seems for iData and the associated mapping tables tickers are all in upper case. This function will therefore strip any leading or ending white space and convert to upper case internally to find match.

`get_countrylist(idata_db='IMF.RES:WEO')`

This is mostly a utility function to see the internal working of mapping country codes. Most users should directly use `translate_ecos_country()`.

Return `idata_utilities.get_dimension_values(idata_db, 'COUNTRY')` for the specified `idata_db`. This is a `pandas.DataFrame` containing the country ISO3 values as index and names as values.

`translate_ecos_country(ecos_country, idata_db='IMF.RES:WEO')`

Translate a given `ecos_country` into the corresponding iData equivalent (typically ISO3). Returns a `str` corresponding to `idata_country`. By default uses `IMF.RES:WEO` as the database to obtain the mapping, but any iData database that has dimension 'COUNTRY' can be used.

Note that this function relies on the [EBV](#) (Enterprise Business Vocabularies) to match the country names used in iData with the IMF codes used in EcOS. Therefore in some cases like country groups, there is no way to find the corresponding matches if the group code is not in the EBV.

`get_idata_data_using_ecos(ecos_db, ecos_country, ecos_series, freq, ecos_counterpart=None)`

Not recommended for use, mostly for testing purposes.

Allows retrieving data from EcOS by specifying `ecos_db`, `ecos_country`, `'ecos_series'`, and `freq`. Optionally specify `ecos_counterpart`.

```
# Get DOT data using get_idata_data_using_ecos
ecos_db = 'ECDATA_DOT'
ecos_series = 'TXG_FOB_USD'
ecos_country = '111'
ecos_counterpart = '193'
freq = 'A'
# This no longer works as EcOS is decommissioned.
df_ecos = ecos_sdmx_utilities.get_ecos_sdmx_data(ecos_db, ecos_country, ecos_series, freq=freq, counterpart=ecos_counterpart)
df_ecos.columns = [c + '_ecos' for c in df_ecos.columns]

# Get from iData
df_idata = get_idata_data_using_ecos(ecos_db, ecos_country, ecos_series, freq, ecos_counterpart=ecos_counterpart)
df_idata.columns = [c + '_idata' for c in df_idata.columns]

# Merge
df = df_ecos.merge(df_idata, left_index=True, right_index=True, how='outer')
print(df)
```

	111TXG_FOB_USD_193.A_ecos	USA.XG_FOB_USD.AUS.A_idata
dates		
1948-01-01	1.140000e+08	1.140000e+08
1949-01-01	1.430000e+08	1.430000e+08
1950-01-01	1.050000e+08	1.050000e+08
1951-01-01	1.830000e+08	1.830000e+08
1952-01-01	1.940000e+08	1.940000e+08
...	...	...
2020-01-01	2.338644e+10	2.338644e+10
2021-01-01	2.646213e+10	2.646213e+10
2022-01-01	3.055162e+10	3.055162e+10
2023-01-01	3.367853e+10	3.367853e+10
2024-01-01	3.459343e+10	3.459343e+10

[77 rows x 2 columns]

3.5 DMXe

The DMXe file format is the new version of DMX and internally is a completely different file type based on sqlite files. Usage is very similar to DMX files, the following functions are available to query and retrieve data, metadata from DMXe files.

Unlike DMX files, DMXe files should be accessible via the `datatools` anywhere as long as Python is installed. Internally the `sqlite3` library is used to access the file, this library should be widely available.

```
get_all_series(dmxfilename, substr=None, debug=False)
```

This function will retrieve all available series codes in a given DMXe file and return a `list`.

For example

```
dmxe_utilities.get_all_series(r'\\was.int.imf.org\ecn\ems_shared\pub\datatools\samples\gab.dmx')
```

will retrieve all series codes from the file `r'\\was.int.imf.org\ecn\ems_shared\pub\datatools\samples\gab.dmx'`.

Several notes of caution are necessary regarding the path to the DMXe file:

1. As the DMXe file is a regular file, you must have access and read permission. Many country desks have DMXe files on their Q drive, users without access to the files will of course not be able to see the contents unless they have access rights.
2. The Q drive of a user depends on which department they are in, so it is recommended to use paths like `\\Data2\apd` instead of `Q:\` so it is clear where the absolute path is, and does not depend on who is running the command.
3. In Windows the path separator is backslash (`\`) which is an escape character⁹ in many languages, including Python. For this reason, Python has what are called “raw strings” which will not use the backslash to be an escape character. This is why the file paths in the above example start with `r`. It is possible to substitute regular slashes and Python will understand the path, for example `//Data2/apd/Data/AUS/DMX/AUS/_MT.dmx` will work, but this may cause confusion since some applications may represent Windows paths in different ways if using forward slashes.

```
>>> serieslist = dmxe_utilities.get_all_series(
    r'\\was.int.imf.org\ecn\ems_shared\pub\datatools\samples\gab.dmx')
>>> print(', '.join(serieslist[:5]))
646BCA, 646BCAXGTO, 646BCAXGTXO, 646BCAXGT, 646BGS
```

```
get_dmx_data(dmxfilename, seriesname, freq=None, scale=None, scale_label=False, debug=False)
```

This function will retrieve data from a given DMXe file and return it as a `pandas.DataFrame`. A simple example is

```
dmxfilename = r'\\was.int.imf.org\ecn\ems_shared\pub\datatools\samples\gab.dmx'
dmxe_utilities.get_dmx_data(dmxfilename, '646BCA')
```

which will retrieve the series 911BF from the above DMXe file. The `seriesname` can be a list of seriescodes, for example

```
dmxe_utilities.get_dmx_data(dmxfilename,
    ['646BCA', '646BCAXGT'])
```

will retrieve both series.

For DMXe files it is possible to store the same series with different frequencies. To distinguish between the frequencies, the parameter `freq` can take on the values "A", "Q", "M", "D" for annual, quarterly, monthly, and daily data. The default value is `None` and if there is only one available frequency, that series is returned. If no `freq` is specified but multiple frequencies exist, a `SystemExit` is raised, while if a `freq` is specified and that frequency series exists, that series is returned.

It is possible to specify a list for `seriesname` so that many series from the same `dmxfilename` can be retrieved at once. The result will be a `pandas.DataFrame` with a column for each series. This will be slightly more efficient than querying each series independently and combining the results afterwards. The `dmxfilename` and `freq` must be common to all series, i.e., it is not possible to get data from multiple DMXe files with one use of this function.

The option `scale` can be set to values of 'dmxe', 'default', 1, 'u', 'k', 'm', 'b', 't'. If the value is 'dmxe' or 'default', the scale stored as metadata will be applied, while 1, u will return the original values, and 'k/m/b/t' will correspond to scaling in thousands, millions, billions, and trillions. If the option `scale_label` is set to `True`, the scale is shown in the columns.

```
get_dmx_metadata(dmxfilename, seriesname=None, substr=None, debug=False)
```

This function will retrieve the metadata of a series or multiple series from a given DMXe file. For example

```
>>> metadata = dmxe_utilities.get_dmx_metadata(
    r'\\was.int.imf.org\ecn\ems_shared\pub\datatools\samples\gab.dmx', '646BCA')
```

⁹For example `\n` is associated with a newline, `\t` is a tab, etc.

```
>>> print(metadata)
      SeriesId  CreatedBy  ...      FiscalLead  LastUpdatedVersion
SeriesCode
646BCA          1  AUmredkar  ...  Start in January          4
```

will retrieve the metadata from the series 646BCA from the above file. The output will be a `pandas.DataFrame` containing available information on the series containing 41 columns, some of which are `CreatedBy`, `CreatedDate`, `UpdatedBy`, `UpdatedDate`, `Source`, `Sector`. These columns can be used to understand which series are of interest.

Note that this metadata depends on the creator of the series inputting this information, otherwise the fields will be empty.

It is also possible to retrieve the metadata for multiple series at once, for example

```
>>> serieslist = ['646BCA', '646BCAXGT']
>>> metadata = dmxe_utilities.get_dmxe_metadata(
    r'\\was.int.imf.org\ecn\ems_shared\pub\datatools\samples\gab.dmxe', serieslist)
>>> print(metadata)
      SeriesId  CreatedBy  ...      FiscalLead  LastUpdatedVersion
SeriesCode
646BCA          1  AUmredkar  ...  Start in January          4
646BCAXGT        4  AUmredkar  ...  Start in January          4
```

will retrieve the metadata for all series in `serieslist` at once and the result will be a row for each series.

If no `seriesname` is specified, the metadata will be retrieved for all series in the DMXe file.

The option `subst` can be used to get metadata only for series containing specific substrings,

```
>>> metadata = dmxe_utilities.get_dmxe_metadata(
    r'\\was.int.imf.org\ecn\ems_shared\pub\datatools\samples\gab.dmxe', substr='BCA')
```

will retrieve metadata for all series containing the substr "BCA". The `substr` can be a list, so that

```
>>> metadata = dmxe_utilities.get_dmxe_metadata(
    r'\\was.int.imf.org\ecn\ems_shared\pub\datatools\samples\gab.dmxe', substr=['BCA', 'X'])
```

will retrieve metadata for series containing both "BCA" and "X". The substrings are case-insensitive, and in the case of DMXe the order matters, so that series of the form "BCA*X*" where * is any character will match, but "X*BCA" will not.

3.6 SQL

There used to be several SQL databases within the Fund that store economic time series data, but most have now been decommissioned. The `sql_utilities` library is legacy code that connected to SQL databases that had similar internal structures to DMX files. The code and documentation of functions below are still included in case it is helpful to connect to SQL databases in the future, but currently there are no use cases.

```
get_all_series((server, dbname, substr=None, debug=False)
```

This function will query a `server` and `dbname` for available series codes. A simple usage would be

```
>>> metadata = sql_utilities.get_all_series('PRDDMXSQL', 'DMX_CF')
```

which will return over 23,000 series as of early 2020.

The option `substr` will narrow down the substrings included in the series codes, for example

```
>>> metadata = sql_utilities.get_all_series('PRDDMXSQL', 'DMX_CF', substr='111')
```

will return only series codes containing "111". The `substr` can be a list,

```
>>> metadata = sql_utilities.get_all_series('PRDDMXSQL', 'DMX_CF',
                                           substr=['111', 'GDP'])
```

will return series codes containing "111" and "GDP". The substrings are case-insensitive, and the order matters, so that in the above example, any series code of the form "111*GDP" where * is any character will match, but series codes of the form "GDP*111" will not.

```
get_sql_data(server, dbname, seriesname, freq=None, scale=None, scale_label=False, debug=False)
```

This function will retrieve data from a given SQL server and database. A simple example is

```
sql_utilities.get_sql_data('PRDDMXSQL', 'DMX_WDI', '111.NY.GDP.COAL.RT.ZS')
```

which will retrieve the series 111.NY.GDP.COAL.RT.ZS} from the SQL serverPRDDMXSQL} and database DMX\WDI}. The `seriesname` can be a list of series codes, for example

```
sql_utilities.get_sql_data('PRDDMXSQL', 'DMX_WDI',
                          ['111.NY.GDP.COAL.RT.ZS', '111.NY.GDP.DISC.KN'])
```

will retrieve both series.

For SQL files it is possible to store the same series with different frequencies. To distinguish between the frequencies, the parameter `freq` can take on the values "A", "Q", "M", for annual, quarterly, and monthly data. The default value is `None` and the function will try to return all available frequencies. If a frequency is specified, the function will return the series with that frequency if it exists, otherwise will return `None`. **Since this behavior is inconsistent with `dmx_utilities`, this behavior will probably change soon.**

It is possible to specify a list for `seriesname` so that many series from the same `server` and `dbname` can be retrieved at once. The result will be a `pandas.DataFrame` with a column for each series. This will be slightly more efficient than querying each series independently and combining the results afterwards. The `server`, `dbname` and `freq` must be common to all series, i.e., it is not possible to get data from multiple SQL servers or databases with one use of this function.

The option `scale` can be set to values of "sql", "default", 1, 'u', 'k', 'm', 'b', 't'. If the value is 'sql' or 'default', the scale stored as metadata will be applied, while 1, u will return the original values, and 'k/m/b/t' will correspond to scaling in thousands, millions, billions, and trillions. If the option `scale_label` is set to `True`, the scale is shown in the columns.

```
get_sql_metadata(server, dbname, seriesname=None, substr=None, standard=False, debug=False)
```

This function will retrieve the metadata of a series or multiple series from a given server and database. For example

```
>>> metadata = sql_utilities.get_sql_metadata('PRDDMXSQL',
                                             'DMX_WDI', '111.NY.GDP.COAL.RT.ZS')
```

```
>>> print(metadata)
```

```
OID Documentation Country_Code Country_Name Subject_Matter Alias ...
Observations_Updated_By UDP_CreatedDate UDP_CreatedBy
UDP_UpdatedDate UDP_UpdatedBy SyncID

111.NY.GDP.COAL.RT.ZS 67448 None 111 United States
None None None None None 2019-04-04 15:13:52.663 IMFNT\TKing None
```

will retrieve the metadata from the series 111.NY.GDP.COAL.RT.ZS from the server PRDDMXSQL and database DMX_WDI. The output will be a `pandas.DataFrame` containing available information on the series containing 41 columns, some of which are `UDP_CreatedBy`, `UDP_CreatedDate`, `UDP_UpdatedBy`, `UDP_UpdatedDate`, `Source`, `Sector`. These columns can be used to understand which series are of interest.

Note that this metadata depends on the creator of the series inputting this information, otherwise the fields will be empty.

It is also possible to retrieve the metadata for multiple series at once, for example

```
>>> serieslist = ['111.NY.GDP.COAL.RT.ZS', '111.NY.GDP.DISC.KN']
>>> metadata = sql_utilities.get_sql_metadata('PRDDMXSQL', 'DMX_WDI', serieslist)
>>> print(metadata)
```

OID	Documentation	Country_Code	Country_Name	Subject_Matter	Alias	...
Observations_Updated_By	UDP_CreatedDate	UDP_CreatedBy	UDP_UpdatedDate	UDP_UpdatedBy	SyncID	
111.NY.GDP.COAL.RT.ZS	67448	None	111	United States	None	None ...
None	None	None	2019-04-04	15:13:52.663	IMFNT\TKing	None
111.NY.GDP.DISC.KN	69728	None	111	United States	None	None ...
None	None	None	2019-04-04	15:13:52.663	IMFNT\TKing	None

will retrieve the metadata for all series in `serieslist` at once and the result will be a row for each series.

If no `seriesname` is specified, the metadata will be retrieved for all series in the SQL server and database specified.

The option `substr` can be used to get metadata only for series containing specific substrings,

```
>>> metadata = sql_utilities.get_sql_metadata('PRDDMXSQL', 'DMX_WDI',
                                              substr='193')
```

will retrieve metadata for all series containing the substr "193". The `substr` can be a list, so that

```
>>> metadata = sql_utilities.get_sql_metadata('PRDDMXSQL', 'DMX_WDI', substr=['193', 'GDP'])
```

will retrieve metadata for series containing both "193" and "GDP". The substrings are case-insensitive, and in the case of DMX the order matters, so that series of the form "193*GDP*" where * is any character will match, but "GDP*193" will not.

Finally, if the option `standard` is set to `True`, only the columns for `Topic` and `UDP_UpdatedDate` will be returned.

3.7 Haver

Haver is a commercial database system that the Fund purchases. It contains approximately 160 databases such as USECON, CANADA, JAPAN, EUNA, INTDAILY and others. Haver also provides some executables which allow programs to query the data and available series.

The data for Haver is stored in proprietary database files prepared by Haver and are kept locally within the Fund. These files are continuously updated to have the latest data and typically Haver has a very quick turnaround between the release of official statistics and updates.

Haver provides its own official **Haver** library which can be accessed with

```
import Haver
```

The code to retrieve Haver was originally written independently at the Fund, but for `imf_datatools` ver 1.2 it was rewritten to rely on the official **Haver** library. Users can check the source code if they wish to change the behavior of the original library.

Documentation can be found at the Haver website's client area at <http://www.haver.com/clientarea/news.html> ¹⁰.

`get_databases()`

¹⁰See <http://www-intranet.imf.org/fundwide/info/EconFinData/Pages/ViewItem.aspx?itemId=2486> for information on how to access the client area.

This function returns all available databases, such as USECON, etc. as a `list`. As of 2021, the IMF is subscribed to 160 databases. Each database can contain several tens of thousands up to several hundreds of thousands of series codes.

`get_all_haver_series(database)` Deprecated in ver 1.2.

`get_haver_data(code, scale=None, eop=False, periods=False, debug=False)`

This function will retrieve data from Haver in the form of `pandas.DataFrames`. If a single series code like `GDP@USECON` is given, the result will be a `pandas.DataFrame` with one column for this data. If multiple series codes are given as a `list`, all valid series codes will be retrieved and each will be a column in the `pandas.DataFrame`.

```
>>> df = haver_utilities.get_haver_data('GDP@USECON')
>>> df[-5:] # print final 5 periods
      GDP@USECON
dates
2018-07-01      20749.8
2018-10-01      20897.8
2019-01-01      21098.8
2019-04-01      21340.3
2019-07-01      21542.5
```

This is an example of retrieving multiple Haver codes at once:

```
# Returns a list of series like 'A11101A', 'A11101AE', ...
serieslist = ['GDP@USECON', 'IP@IP']
df = haver_utilities.get_haver_data(serieslist)
df[-5:] # print final 5 periods
      GDP@USECON  IP@IP
2021-03-01      NaN    99.1237
2021-04-01    22731.4    99.1655
2021-05-01      NaN    99.9870
2021-06-01      NaN   100.1849
2021-07-01      NaN   101.1148
```

If the option `eop` is set to `True`, the dates in the `DataFrame`'s index will be set to the final date of the period:

```
>>> df = haver_utilities.get_haver_data('GDP@USECON', eop=True)
>>> df[-5:] # print final 5 periods
      GDP@USECON
dates
2018-09-30      20749.8
2018-12-31      20897.8
2019-03-31      21098.8
2019-06-30      21340.3
2019-09-30      21542.5
```

If the option `periods` is set to `True`, the dates in the `DataFrame`'s index will be a `PeriodIndex` instead of a `DatetimeIndex`. Note that in this case trying to retrieve multiple series with different frequencies will result in an error.

By default Haver data will be returned in the scale that is shown in the metadata (use the `get_haver_metadata` function to retrieve this). If the option `scale` is set to one of `1`, `k`, `M`, `B`, `T`, the values will be converted to use the specified scales of unit, thousands, millions, billions, trillions.

For example for `GDP@USECON`, the metadata shows that the scale is `Bil.` \$ and the above code examples show that indeed it is in billions. However, when comparing multiple series, each series may have its own units. To undo these, specify `1` (one) as the `scale`:

```
df = haver_utilities.get_haver_data('GDP@USECON', scale=1)
df[-5:] # print final 5 periods
2018-07-01  2.074980e+13
2018-10-01  2.089780e+13
2019-01-01  2.109880e+13
2019-04-01  2.134030e+13
2019-07-01  2.154250e+13
```

If for example T is specified, the values will be converted to trillions:

```
df = haver_utilities.get_haver_data('GDP@USECON', scale='T')
df[-5:] # print final 5 periods
2018-07-01    20.7498
2018-10-01    20.8978
2019-01-01    21.0988
2019-04-01    21.3403
2019-07-01    21.5425
```

get_haver_metadata(code, debug=False)

Get metadata from specified code(s) or a full database. Either a valid code as a `str` or a `list` of codes must be specified. Returns a `pandas.DataFrame` is created with each series code as a row. To retrieve metadata for an entire database, see `get_all_haver_metadata` function.

The Haver database maintains very good metadata, and each series will contain the following 18 columns:

1. database
2. code
3. startdate
4. enddate
5. frequency
6. descriptor
7. numobs
8. datetimemod
9. magnitude
10. decprecision
11. diftype
12. aggtype
13. datatype,
14. group
15. geography1
16. geography2
17. shortsource
18. longsource

get_all_haver_metadata(database)

Retrieve all metadata for a given database. Typically each database has several tens of thousands of series, so this may take a few tens of seconds. Returns a `df` with the same 18 columns as `get_haver_metadata`. for each series contained in `database`.

3.8 World Bank

The World Bank maintains an online web API service ¹¹ This API allows programs to connect and retrieve data, meta-data as soon as it becomes available. The `worldbank_utilities` library connects to this API and parses the available information. Currently the `imf_datatools` focuses on retrieving economic data, but the World Bank also has an API for other data, such as the Climate Data API ¹².

Contact [Econometric Support](#) if other types of data retrieval are necessary and these can be built in.

`get_database_info()`

This function will retrieve information on the available databases within the World Bank API and returns as a `pandas.DataFrame`. The index will be the sorted database names and the columns will be

1. id
2. lastupdated
3. name
4. description
5. dataavailability
6. metadataavailability
7. concepts

See the official documentation on the meaning of these items. As of January 2020 there are 57~databases available.

If the option `debug` is specified, the original downloaded XML data will be attempted to be saved as a local file.

`get_all_worldbank_metadata(debug=False)`

Retrieves metadata on all available series and descriptions and returns as a `'pandas.DataFrame'`. The index will be the series names, and the columns will be

1. name
2. source
3. sourceOrganization
4. unit
5. topic_ids
6. topic_values

See the official documentation on the meaning of these items. As of January 2020 there are 17,328~series codes available.

If the option `debug` is specified, the original downloaded data will be attempted to be saved as a local file.

`get_worldbank_countries(debug=False)`

Retrieves a list of World Bank countries and regions and associated information and saves as a `pandas.DataFrame`. The index will be the ISO 3-letter codes, with columns of

1. name
2. iso2Code
3. region
4. capitalCity
5. incomeLevel
6. latitude
7. longitude

See the official documentation on the meaning of these items. Note that as of January 2020 Kosovo is not assigned a standard code, so while the World Bank uses XK and XXX, this may be different for other organizations. This function

¹¹See <https://datahelpdesk.worldbank.org/knowledgebase/articles/889392-about-the-indicators-api-documentation> for official documentation.

¹²<https://datahelpdesk.worldbank.org/knowledgebase/articles/902061-climate-data-api>

is used by `imf_datatools.get_weo_country_codes()` to combine WEO country code information with the World Bank country information (see Sec. 3.2).

If the option `debug` is specified, the original downloaded XML data and resulting table (csv) will be attempted to be saved as a local file.

`get_worldbank_data(seriesname, country, counterpart=None, freq='A', longformat=False, dataformat='json', debug=False)`

Retrieves data from the World Bank API. The countries are ISO 3-letter codes, such as 'CHN', 'USA', 'JPN', etc., or a list of such countries, such as ['CHN', 'USA', 'JPN']. The special keyword `all` can be used to specify all countries at once. The seriesname can be a series name such as `SP.POP.TOTL` or a list of such series such as ['SP.POP.TOTL', 'AG.AGR.TRAC.NO']. The underlying API cannot actually handle multiple series codes unless the source ID is specified, but this function will internally send each series code and merge the results together.

Examples:

```
# Get population data for China
>>> df = worldbank_utilities.get_worldbank_data('SP.POP.TOTL', 'CHN')
>>> print(df[-5:])
      CHN.SP.POP.TOTL
date
2015-01-01    1.371220e+09
2016-01-01    1.378665e+09
2017-01-01    1.386395e+09
2018-01-01    1.392730e+09
2019-01-01         NaN

# Get multiple series and countries
>>> df = worldbank_utilities.get_worldbank_data(['SP.POP.TOTL', 'AG.AGR.TRAC.NO'],
      ['CHN', 'USA'])
>>> print(df[-5:])
      CHN.SP.POP.TOTL  USA.SP.POP.TOTL  CHN.AG.AGR.TRAC.NO  USA.AG.AGR.TRAC.NO
date
2015-01-01    1.371220e+09    320742673.0         NaN         NaN
2016-01-01    1.378665e+09    323071342.0         NaN         NaN
2017-01-01    1.386395e+09    325147121.0         NaN         NaN
2018-01-01    1.392730e+09    327167434.0         NaN         NaN
2019-01-01         NaN         NaN         NaN         NaN
```

If the option `longformat` is specified as `True`, the data will be returned in a long format so that each row contains a unique combination of `countrycode`, `countryname`, `dates` and each indicator is a column.

```
>>> df = worldbank_utilities.get_worldbank_data(['SP.POP.TOTL', 'AG.AGR.TRAC.NO'],
      ['CHN', 'USA'], long=True)
>>> df.head()
  countrycode countryname  dates  SP.POP.TOTL  AG.AGR.TRAC.NO
0          CHN      China 2019-01-01         NaN         NaN
1          CHN      China 2018-01-01    1.392730e+09         NaN
2          CHN      China 2017-01-01    1.386395e+09         NaN
3          CHN      China 2016-01-01    1.378665e+09         NaN
4          CHN      China 2015-01-01    1.371220e+09         NaN
>>> df.tail()
  countrycode countryname  dates  SP.POP.TOTL  AG.AGR.TRAC.NO
115         USA  United States 1964-01-01    191889000.0    4783000.0
116         USA  United States 1963-01-01    189242000.0    4755000.0
```

117	USA	United States	1962-01-01	186538000.0	4730000.0
118	USA	United States	1961-01-01	183691000.0	4690000.0
119	USA	United States	1960-01-01	180671000.0	NaN

Currently the only supported format for parsing the response is `dataformat='json'`. If the option `debug=True` is specified more output is shown on screen and text files containing the response will be saved in the working directory.

Starting in Nov 2020, some new series containing counterparts were added (see [blog article](#) for details and links to code). By specifying the `counterpart` argument, data can be retrieved from these series. The counterpart country codes are not consistent with the ISO country codes used for other parts of the API. For this reason this function will return the counterpart countries by name and not ISO code. In previous versions, when specifying a counterpart the return format was forced to be in long format, now this is relaxed so series names will be `[countr].[series].[counterpart]`. For the `counterpart` argument, the options "WLD" for "World" and "all" for all counterparts will be useful.

```
>>> df = worldbank_utilities.get_worldbank_data('DT.GPA.DPPG', 'AFG', counterpart='all')
>>> df.tail()
```

	AFG.DT.GPA.DPPG_Asian Dev. Bank	...	AFG.DT.GPA.DPPG_World Bank-IDA
dates		...	
2018-01-01	0.0	...	0.0
2019-01-01	0.0	...	0.0
2020-01-01	0.0	...	0.0
2021-01-01	0.0	...	0.0
2022-01-01	0.0	...	0.0

[5 rows x 19 columns]

```
>>> df = worldbank_utilities.get_worldbank_data('DT.GPA.DPPG', 'AFG', counterpart='all', longformat=True)
>>> df.tail()
```

	dates	country	value	counterpart	series
318	2022-01-01	AFG	0.0	Denmark	DT.GPA.DPPG
319	2022-01-01	AFG	0.0	Croatia	DT.GPA.DPPG
320	2022-01-01	AFG	0.0	China	DT.GPA.DPPG
321	2022-01-01	AFG	0.0	Bulgaria	DT.GPA.DPPG
322	2022-01-01	AFG	0.0	Asian Dev. Bank	DT.GPA.DPPG

In some cases the data with counterpart may have both annual and monthly frequencies. The default is to retrieve the data for annual frequency with option `freq='A'`, this can be changed to `freq='M'` as needed.

```
# Default is annual frequency
>>> df = worldbank_utilities.get_worldbank_data('DT.AMT.BLAT.CD', 'AFG', counterpart='WLD')
>>> df.tail()
```

	AFG.DT.AMT.BLAT.CD_World
dates	
2019-01-01	4400000.0
2020-01-01	500000.0
2023-01-01	50268887.5
2024-01-01	50268887.5
2025-01-01	47615027.5

```
# Specify monthly freq
>>> df = worldbank_utilities.get_worldbank_data('DT.AMT.BLAT.CD', 'AFG', counterpart='WLD', freq='M')
>>> df.tail()
```

	AFG.DT.AMT.BLAT.CD_World
dates	
2025-05-01	0.0

```

2025-06-01          2692006.2
2025-10-01      21394187.5
2025-11-01              0.0
2025-12-01      2134646.2

```

get_worldbank_metadata(seriesname)

Get metadata for a specific series or list of series. The returned result will be a df with an index of series names, and the columns will be

1. name
2. source
3. sourceOrganization
4. unit
5. topic_ids
5. topic_values

See the official documentation on the meaning of these items. This function uses the same API as `get_all_worldbank_metadata` and returns the same result for the specified series.

search_worldbank_metadata(search_str, debug=False)

Searches within the description of series and returns a df of seriesnames and their descriptions that matches `search_str1`. These `search_str1` must be a single 'str1 and the search is case-insensitive.

```

>>> df= worldbank_utilities.search_worldbank_metadata('gdp')
>>> print(df.shape)
(658, 1)
>>> print(df[-5:])

```

	description
SH.XPD.TOTL.ZS	Health expenditure, total (% of GDP)
SL.GDP.PCAP.EM.KD	Labor productivity is used to assess a country...
SL.GDP.PCAP.EM.KD.ZG	GDP per person employed (annual % growth)\nAnn...
SL.GDP.PCAP.EM.XD	GDP per person employed, index (2000 = 100)\nG...
TG.VAL.TOTL.GD.ZS	Merchandise trade (% of GDP)\nMerchandise trad...

get_worldbank_topics(debug=False)

Get available topics within World Bank data and return a df of topic information. As of January 2020 there are 21 topics available. If the option `debug=True` is specified, more output is shown and the information is written out to the working directory.

3.9 BIS

The Bank for International Settlements hosts data and this is accessible through an API, official documentation is at <https://stats.bis.org/api-doc/v1/>. The `bis_utilities` provide functions to access data and metadata, both through the `pandaSDMX` library and also through direct queries to the API.

get_databases()

Get all available datasets. Returns a `pandas.DataFrame` with the database name as the index.

get_dimensions(dataset, refresh=False, debug=False)

Get query dimensions for a given `dataset` and return a list of dimensions to specify the data.

```

dims = bis_utilities.get_dimensions('WS_NA_SEC_DSS')

```

```
['FREQ', 'ADJUSTMENT', 'REF_AREA', 'COUNTERPART_AREA', 'REF_SECTOR', 'COUNTERPART_SECTOR', 'CONSOLIDATION', 'EXPENDITURE', 'UNIT_MEASURE', 'CURRENCY_DENOM', 'VALUATION', 'PRICES', 'TRANSFORMATION', 'CUST_BREAKDOWN']
```

The function retrieves the dimensions and also the available values of each dimension. This is then stored as a dict for this session so that future calls for this information (and calls to `get_dimension_values`) do not repeat the query for the same database. To force an update, use the option `refresh=True`.

```
get_dimension_values(dataset, dimension, refresh=False, debug=False)
```

Get available values for a given `dataset` and `dimension`. Returns a `pandas.DataFrame` with the index set to available values.

```
vals = bis_utilities.get_dimension_values('WS_NA_SEC_DSS', 'REF_AREA')
```

	name	parent
CL_BIS_IF_REF_AREA		
00	Others	CL_BIS_IF_REF_AREA
11	Technical residual (Non-residents / Cross-border)	CL_BIS_IF_REF_AREA
1A	US banks in offshore centres	CL_BIS_IF_REF_AREA
1B	International banking facilities	CL_BIS_IF_REF_AREA
1C	International organisations	CL_BIS_IF_REF_AREA
...	...	...
ZA	South Africa	CL_BIS_IF_REF_AREA
ZM	Zambia	CL_BIS_IF_REF_AREA
ZR	Zaire	CL_BIS_IF_REF_AREA
ZW	Zimbabwe	CL_BIS_IF_REF_AREA
_Z	Not applicable	CL_BIS_IF_REF_AREA

```
[432 rows x 2 columns]
```

```
get_bis_sdmx_data(db, key={}, params={}, longformat=False, debug=False)
```

Get BIS data using `pandaSDMX`. By default will get all data within a given dataset, otherwise specify dict for filtering params with `key`. The dict `params` allows specifying `startPeriod`, `endPeriod`.

If not `longformat` (default), a `pandas.DataFrames` will be returned with the dates as the index. If `longformat=True`, each dimension of the dataset will be its own column.

```
db = 'WS_NA_SEC_DSS'
```

```
key = {'FREQ' : 'Q',
       'ADJUSTMENT' : 'N',
       'REF_AREA' : 'AR',
       'COUNTERPART_AREA' : 'XW',
       'REF_SECTOR' : 'S1+S13',
       'COUNTERPART_SECTOR' : 'S1',
       'CONSOLIDATION' : 'N',
       'ACCOUNTING_ENTRY' : 'L',
       'STO' : 'LE',
       'INSTR_ASSET' : 'F3',
       'MATURITY' : 'T',
       'EXPENDITURE' : '_Z',
       'UNIT_MEASURE' : 'USD',
       'CURRENCY_DENOM' : '_T',
       'VALUATION' : 'N',
       'PRICES' : 'V',
```

```

'TRANSFORMATION' : 'N',
'CUST_BREAKDOWN' : '_T'}
params = {'startPeriod' : 2000, 'endPeriod' : '2023-06'}

df = get_bis_sdmx_data(db, key=key, params=params, debug=False)

      Q.N.AR.XW.S1.S1.N.L.LE.F3.T._Z.USD._T.N.V.N._T  Q.N.AR.XW.S13.S1.N.L.LE.F3.T._Z.USD._T.N.V.N._T
dates
2018-04-01      272.276059      NaN
2018-07-01      240.062311      NaN
2018-10-01      239.874817      NaN
2019-01-01      236.950934      NaN
2019-04-01      241.963724      NaN
2019-07-01      208.480027      NaN
2019-10-01      200.195167      NaN
2020-01-01      203.900941      NaN
2020-04-01      208.396845      NaN
2020-07-01      214.932335      NaN
2020-10-01      214.533246      NaN
2021-01-01      233.672621      192.210904
2021-04-01      244.217507      199.809332
2021-07-01      249.627316      204.964572
2021-10-01      255.009723      212.250929
2022-01-01      279.992746      220.581546
2022-04-01      296.087258      224.361338
2022-07-01      308.310803      233.198126
2022-10-01      311.728939      238.958269
2023-01-01      315.429912      242.940868
2023-04-01      327.583617      251.903298

# Get data in long format
df = get_bis_sdmx_data(db, key=key, params=params, longformat=True, debug=False)

```

```
get_bis_data(db, code, start=None, end=None, longformat=False)
```

Get BIS data using direct queries to the API, not through SDMX. Specify the database name `db` as well as a `str` code that specifies the series code(s). This will take a form like 'Q.N..XW.S1+S11+S12+S13.S1.N.L.LE.F3.T._Z.USD._T.N.V.N._T+C01+C02' where each dimension is separated by a `..` To specify multiple values, separate by `+` as in the above for `S1+S11+S12+S13`, and to allow all values, leave empty as in the above third dimension.

If `longformat` is specified, each dimension of the query will be put in its own column.

```

db = 'WS_NA_SEC_DSS'
code = 'Q.N.AR.XW.S1+S13.S1.N.L.LE.F3.T._Z.USD._T.N.V.N._T'
df = get_bis_data(db, code)

      Q.N.AR.XW.S1.S1.N.L.LE.F3.T._Z.USD._T.N.V.N._T  Q.N.AR.XW.S13.S1.N.L.LE.F3.T._Z.USD._T.N.V.N._T
dates
2018-04-01      272.276059      NaN
2018-07-01      240.062311      NaN
2018-10-01      239.874817      NaN
2019-01-01      236.950934      NaN
2019-04-01      241.963724      NaN
2019-07-01      208.480027      NaN

```

2019-10-01	200.195167		NaN
2020-01-01	203.900941		NaN
2020-04-01	208.396845		NaN
2020-07-01	214.932335		NaN
2020-10-01	214.533246		NaN
2021-01-01	233.672621	192.210904	
2021-04-01	244.217507	199.809332	
2021-07-01	249.627316	204.964572	
2021-10-01	255.009723	212.250929	
2022-01-01	279.992746	220.581546	
2022-04-01	296.087258	224.361338	
2022-07-01	308.310803	233.198126	
2022-10-01	311.728939	238.958269	
2023-01-01	315.429912	242.940868	
2023-04-01	327.583617	251.903298	
2023-07-01	305.675820	234.388816	
2023-10-01	236.740514	207.440018	
2024-01-01	267.115846	237.911544	

```

# Get data in long format
df = get_bis_data(db, code, longformat=True)

```

3.10 Troubleshooting

For all resources, the `imf_datatools` return data only if you have access to it. Please consult the appropriate parties for these permissions. The `imf_datatools` and its authors do not grant permissions to anything. In case a resource does not return data, check that the data can be accessed using conventional methods such as the various Excel add-ins that are available within the Fund. If the data is not retrieved it may be a permission issue. For trouble accessing data with:

1. Haver

Use the Haver DLXVG3 tool (available from the Software Center) to check that series codes exist. There is no user-dependent permission for Haver.

2. World Bank

Check the World Bank website that data of interest is available. There are no user-dependent permissions.

Chapter 4

Further Data Retrieval and Manipulation

4.1 Writing to DMXe Files

A new feature as of Jun 2024 is the ability to write to DMXe files. This is done by using the same executable files that Excel uses when running the DMXe add-in, so that the behavior should be exactly the same as using through Excel.

To install the DMXe writer, make sure the library `pythonnet` is installed. Check with the command `import pythonnet` in a python session, and if this gives an error, run

```
pip install pythonnet
```

in a Command Prompt window.

The commands can also be called from Stata via the command `dmxewriter`. Use the Stata command

```
help dmxewriter
```

to get further info and examples, and this is also shown in Sec. 5.2.4. A very simple example of Stata is shown below.

```
// Get annual data from iData.
idatause data, database(IMF.RES:WEO) query(USA+AUS+NZL.NGDP+PCPI.A) f(idata_A, replace)

// Write data to a DMXe file.
dmxewriter data , dmxepath(C:/temp/dmxewriter_example.dmx) frame(idata_A) indicator(USA_NGDP_A, AUS_PCPI_A)

// Modify column Descriptor. Descriptor can take any string value.
dmxewriter metadata, dmxepath(C:/temp/dmxewriter_example.dmx) indicator(USA.NGDP.A) column(Descriptor) value

// Delete values of indicators from a DMXe file.
dmxewriter delete, dmxepath(C:/temp/dmxewriter_example.dmx) indicator(USA.NGDP.A) freq(A)
```

Below the available commands and examples are provided.

```
save_dmx_data(outfilename, df, freq=None, datecol=None, debug=False)
```

Save DMXe data for a `pandas.DataFrame` `df` into a DMXe file `outfilename`. `df` should be in the standard format for `imf_datatools` with an `Index` corresponding to the dates/periods, and each column providing a different series. If `freq=None` is given, the frequency of each column is determined by the `imf_datatools` and saved in the DMXe file, otherwise specify a frequency like A, Q, M, D and B. For cases (such as when using through R) when an index is not given, specify the column for dates using the argument `datecol`.

```
import imf_datatools
from imf_datatools import dmxe_writer_utilities
# Get data
df = imf_datatools.get_haver_data('IP@IP')

# Save to DMXe file
filename = 'out.dmx'
dmxe_writer_utilities.save_dmx_data(filename, df)
```

```
save_dmx_metadata(outfilename, seriescode, dict_metadata, debug=False)
```

Save DMXe metadata for a given series `seriescode` into a DMXe file `outfilename`. The attributes are stored in `dict_metadata` which is a dictionary between the metadata header like “Notes” and the corresponding text. Note that DMXe places restrictions on some metadata headers so that only a pre-specified set of values are valid, and other fields like “Country_Code” are automatically generated so cannot be specified. See the [documentation](#) for details.

```
import imf_datatools
from imf_datatools import dmxe_writer_utilities
# Get data
df = imf_datatools.get_haver_data('IP@IP')

# Save to DMXe file
filename = 'out.dmx'
dmxe_writer_utilities.save_dmx_data(filename, df)

# Update metadata
dict_metadata = {'Notes' : 'Downloaded from Haver'}
dmxe_writer_utilities.save_dmx_metadata(filename, 'IP@IP', dict_metadata)
```

```
delete_dmx_data(filename, series, freq, debug=False)
```

Delete a `series` from the DMXe file `filename`. `freq` can take on a frequency like A, Q, M, D, and two special cases:

1. `all` will delete all frequencies.
2. `series` will delete the series itself.

```
rename_dmx_data(filename, series, newseries, debug=False):
```

Rename a `series` in the DMXe file `filename` to another name `newseries`.

```
check_logs(filename, debug=False)
```

Check and return how many rows of access logs there are in a DMXe file `filename`.

```
delete_logs(filename)
```

Delete the access logs in the DMXe file `filename`. These access logs can take up substantial space, especially after batch operations, so cleaning periodically is recommended.

```
read_dmx_data(infilename, series, freq=None, debug=False)
```

Utility function to retrieve DMXe data using same executables as used in Excel add-in. Same functionality as `dmxe_utilities.get_dmx_data()`, but also allows calculation based on expressions. If `freq` is `None` and multiple frequencies exist, the lowest available frequency is returned.

An example of reading the data as-is and applying a DMXe calculation is shown below. For details of DMXe functions see the documentation available at the [DMXe website](#).

```
import imf_datatools
from imf_datatools import dmxe_writer_utilities
```

```

# Get data
df = imf_datatools.get_haver_data('IP@IP')

# Save to DMXe file
filename = 'out.dmx'
dmxe_writer_utilities.save_dmx_data(filename, df)

# Read out data as-is
df2 = dmx_writer_utilities.read_dmx_data(filename, 'IP@IP')

# Do calculation using DMXe.
# In this example we calculate the %yoy.
# The additional parameters are to specify the lag and to use percentage.
df_yoy = dmx_writer_utilities.read_dmx_data(filename, 'calc(TsPCH(IP@IP, 12, 1))')

# Calculate moving average over 12 months
df_mavg = dmx_writer_utilities.read_dmx_data(filename, 'calc(TsMAvg(IP@IP, 12))')

# If changing the frequency of the output from the original, specify the freq
df_q = dmx_writer_utilities.read_dmx_data(filename, 'calc(TsConsolidate(IP@IP, average))', freq='Q')

set_dmx_libdir(corepath, mappingpath)

```

The default path for the necessary DMXe libraries is C:\Program Files\International Monetary Fund\DMXe\Core. If for whatever reason this path needs to be changed set using the arguments `corepath` and `mappingpath`.

This should only be relevant on the servers where the DMXe libraries are not installed by default to a fixed location. To run on the server, copy the two folders Core and Template under C:\Program Files\International Monetary Fund\DMXe to the server for example to E:\data\[user]\DMXe', then edit the file `dmx_writer_utilities.py` which you are importing to have the lines

```

# Add this line to dmx_writer_utilities.py before
# set_dmx_libdir() is called.
DMXECOREDIR = r'E:\data\kmoriya\DMXe\Core'

# This line already exists.
MAPPINGFILE = DMXECOREDIR + '/../Template/Mapping.dmx'

```

4.2 Data Retrieval Scripts

Each library in section 3 retrieves data from a single source like iData, DMXe, or Haver. For Python users it should be relatively easy to write a script that will get the data of interest from these sources. However, for some users it may be convenient to be able to specify the necessary data in an Excel file and have a single command retrieve the data for them.

For this purpose there is a script `datacollector.py` in the `imf_datatools/scripts` folder. The complete documentation is [here](#). The script requires two arguments, the input Excel file name and the output Excel file name. The simplest use case would be

```
python datacollector.py path/to/input.xlsx output/path/output.xlsx
```

which will read in the contents of `path/to/input.xlsx` and write out to `output/path/output.xlsx`. These paths can be relative to where the command is run, or an absolute path like:

```
C:\Users\jdoe\Desktop\input.xlsx.
```

Note that if the path contains spaces, the entire path must be put in double quotes:

```
python datacollector.py "input (1).xlsx" "output/path/output file.xlsx"
```

The output file will be overwritten each time the script is run. Users should be careful not to overwrite existing files.

In the case that the input Excel file contains multiple sheets, the sheet can be specified via `input.xlsx::sheet1` so that in this case the sheet named `sheet1` will be used.

The input Excel file must have the following structure.

1. Valid input rows will specify a series to retrieve. Column A will be an optional *alias* of the series, which will become the column name for that series. The alias cannot contain special characters or start with a numerical value. Column B will be the resource name, which can be one of `iData`, `DMXe`, `SQL`, `Haver`, `World Bank`. This column is case-insensitive, so `dmxe` for example will work. In column C onwards are the necessary parameters to specify the series, and these follow the `get_[resource]_data` functions explained in section 3. For example for `iData` the corresponding function would be `get_idata_data` (see Sec. 3.3 so columns C, D, E would be the database name, country code, and seriescode. For this script, all input rows can specify only one series.
2. A further option is to **apply calculations** to series. To do this, use a row to specify the alias (necessary for calculation rows) and the expression. For example, if series with aliases `gdp_USA` and `gdp_AUS` have been defined in rows after that calculations using those aliases can be specified:

	A	B	C	D	E
23					
24	# WEO data				
25	gdp_USA	EcOS	WEO_WEO_PUBLISHED	111	NGDP
26	gdp_AUS	EcOS	WEO_WEO_PUBLISHED	193	NGDP
45	pop_gdpr_USA	pop([gdp_USA])	# Apply period-over-period % change		
46	gdp_USA_AUS	[gdp_USA] + [gdp_AUS]	# Sum two series		
47	pop_gdp_USA_AUS	yoy([gdp_USA] + [gdp_AUS])	# Apply year-over-year % change to sum		
48					

In this example the calculations are

1. period-over-period % change of a series (applying function `calc_pop`, . All functions from `dataframe utilities` can be applied using the name of the function but without the `calc_` part. Each alias of a series must be put in square brackets like `pop(gdp_USA)`.
2. Sum of two series. Similarly, the sum, difference, product and division of two series can be done.
3. Apply the year-over-year % change of a series (applyin function `calc_yoy`) to the sum of two series.

In this way, almost any new series can be derived from series that are already defined.

3. If a hash mark (#) is specified at the beginning of a cell, that cell and all further cells in that row will not be included in the input. This is an easy way to add comments to the input sheet. Empty rows are also ignored.
4. If in column A `END` is specified (case-insensitive), further input rows are ignored. This is an easy way to cut off a long input and test it up to a certain row.

The example file `input_dc.xlsx` which is included in the `imf_datatools/scripts` folder shows examples of retrieving data from various sources and calculations among them.

The script takes in several options, listed below. To see the available options, try `python datacollector.py -h` or `python datacollector.py --help`.

1. `-f` or `--freq` Allows specifying the frequency of the data and ensuring that all retrieved data is of that frequency. If data is specified that does not match this frequency, an error is raised. Specify one of `A`, `Q`, `M`, `D` for annual, quarterly, monthly, or daily data, respectively.

2. `-s` or `--separate` By default all data will be merged together regardless of frequency into one df. This is possible because for each frequency, the data takes on the date of the first day of each period. If this option is specified, the results will be written out in separate sheets for each frequency.
3. `-l` or `--log` Allows a log file to be created showing what the program did. This may be useful if trying to understand why the program is not working.
4. `-v` or `--verbose` Show more output of what the program is doing.

Examples:

```
# Specify input sheet as data
python datacollector.py input.xlsx::data output.xlsx

# Retrieve data from 1990 to 2015 only
python datacollector.py input.xlsx output.xlsx -t 1990,2015

# Retrieve only annual data, raise error if non-annual data is specified
python datacollector.py input.xlsx output.xlsx -f A

# Specify output is split by frequency
python datacollector.py input.xlsx output.xlsx -s

# Specify output sheet and that output split by frequency.
# In this case each sheet name will have the frequency appended
python datacollector.py input.xlsx output.xlsx::out -s

# Create log file. This log file will be in a folder log
# where the user is running the code
python datacollector.py input.xlsx output.xlsx::out -s -l
```

4.3 dataframe_utilities

The `imf_datatools` comes with a utility library `dataframe_utilities.py` which contains many utility functions to apply calculations such as percent changes and frequency conversions to `pandas.DataFrames` created by the `datatools`. Users familiar with the `pandas` library can of course write their own versions of these functions on the fly but having a common set of functions can be useful.

Below each available function is documented with available options.

`merge_dfs(iterable_dfs, how='outer')`

This function takes in an iterable of `pandas.DataFrames` and merges them into one `pandas.DataFrame`. It is assumed that each df has a `pandas.Timestamp` as an index and by default the function will do an outer join so that any date that was in the original index values will be in the output. Since the function operates on `pandas.Timestamps`, it is possible to merge dfs of different frequencies, then apply `ffill` and other functions so that all rows have a value.

```
# merge two quarterly series
df = haver_utilities.get_haver_data('GDP@USECON')
df2 = haver_utilities.get_haver_data('J025PCW@EUNA')
df3 = dataframe_utilities.merge_dfs([df, df2])
print(df3[-3:])
           J025PCW@EUNA  GDP@USECON
dates
2019-01-01           0.20      21098.8
```

2019-04-01	0.12	21340.3
2019-07-01	0.27	21542.5

```
# merge montly and quarterly series
df = haver_utilities.get_haver_data('IP@IP') # monthly
df2 = haver_utilities.get_haver_data('GDP@USECON') # quarterly
df3 = dataframe_utilities.merge_dfs([df, df2])
# quarterly series only has values at beginning of each quarter,
# 19Q4 value was not available at this time.
print(df3[-8:])
```

	GDP@USECON	IP@IP
dates		
2019-04-01	21340.3	108.9888
2019-05-01	NaN	109.2264
2019-06-01	NaN	109.2774
2019-07-01	21542.5	109.0852
2019-08-01	NaN	109.9634
2019-09-01	NaN	109.4437
2019-10-01	NaN	108.8532
2019-11-01	NaN	109.7573
2019-12-01	NaN	109.4330

Functions to calculate time series

These functions apply transformations that are common to time series, such as percent changes, shifts, and differences. The output of these functions is a `pandas.DataFrame` (or a `pandas.Series` if the input was a `pandas.Series`), so the functions can be chained together to calculate various expressions such as

- %yoy of a moving 3-month average
`calc_yoy(calc_rolling(df, method='mean'))`
- contribution of one series to the growth of another
`calc_diff(df, 4) / calc_shift(df2, 4)`
- Calculate an index series normalized to 100 at a given period for the ratio of two series
`calc_rebase(df[col1] / df2[col2], '2020-01')` # will set the value at 2020 Jan to be 100 (default)

The functions assume by default that there is an index for the input `DataFrame` that is a `pd.DatetimeIndex` with name `dates`, but the option `indexcol` allows specifying a column to use as the date index. This is useful for example if calling these functions from R where the index information may be lost.

1. `calc_pop(df, nshift=1, datecol=None)`

Calculates percent change from previous period (period-over-period). Default is to calculate percent change from `nshift=1` period ago, but this can be adjusted.

2. `calc_yoy(df, datecol=None)`

Calculates percentage change from previous year (year-over-year).

3. `calc_shift(df, n=1, datecol=None)`

Shifts values against index of `df`. Positive values of shift result in values moving to future dates, while negative shift values move values to past dates. Original dates in the index where values no longer exist will be given `NaN` values. This function can be useful when combined with other functions to calculate contributions to growth and other values.

```
# Get monthly data
df = haver_utilities.get_haver_data('IP@IP')
```

```

# First values
print(df[:3])
      IP@IP
dates
1921-01-01  4.0985
1921-02-01  4.0155
1921-03-01  3.9047
# Final values
print(df[-3:])
      IP@IP
dates
2019-10-01  108.8532
2019-11-01  109.7573
2019-12-01  109.4330
# Apply shift (default of 1).
# Original first month's value has become NaN
print(df.shift()[:3])
      IP@IP
dates
1921-01-01    NaN
1921-02-01  4.0985
1921-03-01  4.0155
# Final period has been added
print(df.shift()[-3:])
      IP@IP
dates
2019-11-01  108.8532
2019-12-01  109.7573
2020-01-01  109.4330

```

4. `calc_diff(df, shifts=1, datecol=None)`

Calculates the difference of a series for a number of shifts. For positive shifts of `n` the difference is with `n` previous periods.

5. `calc_ana(df, datecol=None)`

Calculate annualized percent change with formula

$$y_{ana}(t) = 100 \times \left(\left(\frac{y(t)}{y(t-1)} \right)^p - 1 \right)$$

where $y(t)$ is the original time series and p is 4 for quarterly series and 12 for monthly series.

6. `calc_rolling(df, nperiods=1, method='mean', datecol=None)`

Calculate rolling mean or sum of a series. Specify number of periods and method with `how='mean'` or `how='sum'`. Result will have the rolling mean or sum of `nperiods` ending at that period. If there are less than `nperiods` before that period, values will be NaN.

```

# Monthly data
df = haver_utilities.get_haver_data('IP@IP')
print(df[-3:])
      IP@IP
dates
2019-10-01  108.8532

```

```

2019-11-01    109.7573
2019-12-01    109.4330
# Calculate rolling mean of 3 months
print(calc_rolling(df, 3, 'mean')[-3:])
dates
2019-10-01    109.420100
2019-11-01    109.351400
2019-12-01    109.347833
# Calculate rolling sum of 3 months
print(calc_rolling(df, 3, 'sum')[-3:])
      IP@IP
dates
2019-10-01    328.2603
2019-11-01    328.0542
2019-12-01    328.0435
# First 2 periods will become NaN
print(calc_rolling(df, 3, 'sum')[:3])
      IP@IP
dates
1921-01-01      NaN
1921-02-01      NaN
1921-03-01    12.0187

```

7. calc_ytd(df, start='Jan', datecol=None, debug=False)

Calculate year-to-date sums for a given series. By default, for monthly and quarterly series, the output will be the cumulative sum of the series starting in January or Q1 of that year. The starting period can be changed with the option `start`, for monthly series specifying `start='Apr'` will give the cumulative sum of each year starting in April.

```

df = haver_utilities.get_haver_data('IP@IP')
print(df[-3:])
      IP@IP
dates
2019-10-01    108.8532
2019-11-01    109.7573
2019-12-01    109.4330
# Calculate ytd starting in Jan
print(calc_ytd(df)[-3:])
      IP@IP
dates
2019-10-01   1094.2008
2019-11-01   1203.9581
2019-12-01   1313.3911
# Calculate ytd starting in Oct
print(calc_ytd(df, start='Oct')[-3:])
      IP@IP
dates
2019-10-01    108.8532
2019-11-01    218.6105
2019-12-01    328.0435
# quarterly series
df_q = haver_utilities.get_haver_data('GDP@USECON')
# Calculate ytd starting in Q1
print(calc_ytd(df_q)[-5:])

```

```

GDP@USECON
dates
2018-07-01    61423.2
2018-10-01    82321.0
2019-01-01    21098.8
2019-04-01    42439.1
2019-07-01    63981.6
# Calculate ytd starting in Q2
print(calc_ytd(df_q, start='Q2')[-5:])
GDP@USECON
dates
2018-07-01    41260.0
2018-10-01    62157.8
2019-01-01    83256.6
2019-04-01    21340.3
2019-07-01    42882.8

```

8. `calc_diffytd(df, start='Jan', datecol=None, debug=False)`

Undo year-to-date sums for a given series. By default, for monthly and quarterly series, the output will take January or Q1 of that year to be the starting period. The starting period can be changed with the option `start`, for monthly series specifying `start='Apr'` will undo the cumulative sum for each year starting in April.

```

# A series that already has YTD applied
df = haver_utilities.get_haver_data('Y238IXD@EMERGELA')
print(df[-6:])
Y238IXD@EMERGELA
dates
2020-10-01    9531.8
2020-11-01   10586.9
2020-12-01   11625.7
2021-01-01    950.5
2021-02-01    2050.4
2021-03-01    3310.5
# Unravel the YTD starting in Jan
print(calc_diffytd(df)[-6:])
Y238IXD@EMERGELA
dates
2020-10-01    1046.8
2020-11-01    1055.1
2020-12-01    1038.8
2021-01-01     950.5
2021-02-01   1099.9
2021-03-01   1260.1

```

If the same start period is used, applying `ytd` and `diffytd` in succession should yield the original series.

9. `calc_pow(df, p=1, datecol=None)`

Calculate power of a series for any real number.

10. `calc_rebase(df, startdatestr, col=None, rebase_val=100, datecol=None, enddatestr=None, debug=False)`

Rebase a time series to a given period or range of periods and value. By default the value for the date specified by `startdatestr` will be rebased to 100, but this value can be changed with option `rebase_val`. If `enddatestr` is provided, the normalization will be the average value between `startdatestr` and `enddatestr`.

```

df = haver_utilities.get_haver_data('IP@IP')
print(df[-3:])
      IP@IP
dates
2023-08-01  103.2927
2023-09-01  103.3643
2023-10-01  102.7081

# Rebase at 2023-09
print(calc_rebase(df, '2023-09')[-3:])
      IP@IP
dates
2023-08-01    99.930730
2023-09-01   100.000000
2023-10-01    99.365158

# Rebase to different value
print(calc_rebase(df, '2023-09', rebase_val=1)[-3:])
      IP@IP
dates
2023-08-01    0.999307
2023-09-01    1.000000
2023-10-01    0.993652

# Rebase to average over 2010-2019
print(calc_rebase(df, '2010-01', enddatestr='2019-12')[-3:])
      IP@IP
dates
2023-08-01   104.299694
2023-09-01   104.371992
2023-10-01   103.709395

# Print the mean between 2010-2019 to confirm.
print(calc_rebase(df, '2010-01', enddatestr='2019-12').loc['2010' : '2019'].mean())
IP@IP    100.0
dtype: float64

```

```
11. calc_ffill(df, nperiods=1, datecol=None, debug=False)
```

Forward-fill data for `nperiods` using the latest value. This is *NOT* the same as the `pandas.DataFrame ffill` function, it is not for filling NA values but for extending the `DataFrame` by `nperiods` with the last available value.

```
12. calc_vol(df, nperiods, datecol=None, debug=False)
```

Calculate the volatility of a `DataFrame` for `nperiods`. Volatility is defined as standard deviation of the last `nperiods` divided by the average in that period.

```
13. calc_unpop(df, init_val=100, ffill=True, datecol=None, debug=False)
```

Undo a %pop calculation, useful if the series was provided as such. `init_val` specifies the initial period value.

Since the calculation is a cumulative product of series value, if there are any NA values in the input `df` the values will all be NA after that value. By default if any NA values exist after reindexing to a full index, the function will fill missing values with NA. If this behavior is not wanted and to return only the last part of the series without any NA values, set `ffill=False`.

14. `calc_cumsum(df, datecol=None, debug=False)`

Calculate the simple cumulative sum of a `DataFrame` or `Series`.

Functions to apply frequency conversion

The below functions can be used to convert the frequency of a time series to another. As the `imf_datatools` will always return the index of a `DataFrame` as the beginning date of each period, it is necessary to use these functions when combining time series with different frequencies.

For *downsampling*, where the new frequency is lower than the original, the basic choices are to take the `mean` or `sum` of the subperiods that create the new period, or to take the `first` or `last` available values within the new period. In the case that output for only complete periods are wanted, specify `complete=True` and this will remove incomplete periods. The `complete` option is not available for functions `calc_d2m` and `calc_w2m` since it can be difficult to determine whether a day or week completes a month or not.

For *upsampling*, where the new frequency is higher than the original, the two choices are to take the `level` so that the original value for a period is given to each subperiod that is included, or the `'mean'` where the original value of a period is distributed evenly among the subperiods.

1. `calc_d2w(df, method='mean', day='Sun', datecol=None, debug=False)`

Convert a daily time series to weekly. The default will be to take the mean over the week and set the index to Sundays. Options are `mean`, `average`, `sum`, `first`, `last`.

2. `calc_d2m(df, method='mean', datecol=None)`

Convert a daily time series to monthly. By default will take the mean over each month. Other options are `sum`, `first`, `last`.

3. `calc_d2q(df, method='mean', datecol=None)`

Convert a daily time series to quarterly. By default will take the mean over each quarter. Other options are `sum`, `first`, `last`.

4. `calc_w2m(df, method='mean', datecol=None)`

Convert a weekly time series to monthly. By default will take the mean over each month. Other options are `sum`, `first`, `last`.

5. `calc_m2q(df, method='mean', complete=False, datecol=None)`

Convert a monthly time series to quarterly. By default will take the mean over each quarter. Other options are `sum`, `first`, `last`, `middle`. If `middle` is specified, the value of the middle month is used. If `complete=True` is specified, only values for complete quarters are returned.

6. `calc_m2a(df, method='mean', complete=False, datecol=None)`

Convert a monthly time series to annual. By default will take the mean over each year. Other options are `sum`, `first`, `last`, `ytdcurrent`, or the first three characters of a month name like `Jan` or `Apr` to return the value of that month for each year. If `ytdcurrent` is specified, the year-to-date sum is calculated for each year ending at the last month of the series. That is, if the series ends in August, the year-to-date sum from the previous year's July to the current year's August is calculated for each year. If `complete=True` is specified, only values for complete years are returned.

7. `calc_q2a(df, method='mean', complete=False, datecol=None)`

Convert a quarterly time series to annual. By default will take the mean over each year. Other options are `sum`, `first`, `last`, or `Q1`, `Q3`, etc., to return the value of that quarter for each year. If `complete=True` is specified, only values for complete years are returned.

8. `calc_q2m(df, method='level', datecol=None)`

Convert a quarterly series to monthly, defaults to `level` so that all months within a quarter are given the original quarter's value, or can also specify `mean` which will split the quarter value by 3.

```
9. calc_a2m(df, method='level', datecol=None)
```

Convert an annual series to monthly, defaults to `level` so that all months within a year are given the original year's value, or can also specify `mean` which will split the annual value by 12.

```
10. calc_a2q(df, method='level', datecol=None)
```

Convert an annual series to quarterly, defaults to `level` so that all quarters within a year are given the original year's value, or can also specify `mean` which will split the annual value by 4.

Chapter 5

Using with other Applications

For applications (programs or language) that can use system commands, the `imf_datatools` can be called as Python scripts to create output that the application can read, for example Excel or csv files using the data retrieval scripts from Sec. 4.2. Applications like MatLab and Mathematica could use the `imf_datatools` in this way, or could be used in workflow scripts. Note that it is possible to have scheduled tasks that can periodically update data which can then be fed into various dashboards, reports, and other systems so that the whole process can be automated from data retrieval to output.

Below are some applications where the `imf_datatools` can be used directly without incurring system commands and writing out/reading in the output.

5.1 R

The `imf_datatools` can be used to get data directly loaded into a R data frame without having to write out and read in through an intermediate format. See the installation instructions in Chapter 2 on how to invoke the `imf_datatools` in R.

The following sections show some examples of how data retrieval can be done directly within R. This is not meant as a comprehensive guide about the `imf_datatools`, but just examples of how Python commands will be translated into their R counterparts.

Warning for timezones R

The table of data retrieved in Python is automatically translated into an R data frame by the `reticulate` package. However, when translating columns containing datetime values from Python to R, the `reticulate` package will automatically interpret the datetimes as being in UTC time and convert to local time. It seems that with the new version of Python 3.10.9 and R 4.4.0 as of June 2024, this applies to index datetimes as well.

The simplest solution is to set the local timezone to UTC with the command

```
Sys.setenv(TZ="UTC")
```

Be warned however that if the local time is used by `Sys.time()` and other commands to show for example the time that a document was created, this will also be shown in UTC.

```
Sys.time()
# "2021-05-11 21:44:22 EDT"

# Set time zone to UTC
Sys.setenv(TZ="UTC")
```

```
# Time is now shown in UTC
Sys.time()
# "2021-05-12 01:45:58 UTC"
```

In cases where `longformat=TRUE` is used, that is, all dates are contained in columns of the DataFrame and not in the index, there is the option to use the function `with_tz` from the `lubridate` package to force the times to be interpreted as UTC.

An example of the problem is shown below:

```
# In longformat dates are now converted to local time zone!
tail(imf_datatools$get_idata_data("IMF.STA:IMTS", "USA.XG_FOB_USD.G001.Q", longformat=TRUE))
#      COUNTRY  INDICATOR COUNTERPART_COUNTRY FREQUENCY      dates      OBS_VALUE
# 259      USA XG_FOB_USD                G001          Q 2024-06-30 20:00:00 520712438379
# 260      USA XG_FOB_USD                G001          Q 2024-09-30 20:00:00 518048376275
# 261      USA XG_FOB_USD                G001          Q 2024-12-31 19:00:00 523834888355
# 262      USA XG_FOB_USD                G001          Q 2025-03-31 20:00:00 553002126621
# 263      USA XG_FOB_USD                G001          Q 2025-06-30 20:00:00 542584602051
# 264      USA XG_FOB_USD                G001          Q 2025-09-30 20:00:00 565784230444
```

In the below the `with_tz` function is applied so the dates are correctly set:

```
library(lubridate)

# Now times are correct:
tail(with_tz(imf_datatools$get_idata_data("IMF.STA:IMTS", "USA.XG_FOB_USD.G001.Q", longformat=TRUE), tzzone="U
#      COUNTRY  INDICATOR COUNTERPART_COUNTRY FREQUENCY      dates      OBS_VALUE
# 259      USA XG_FOB_USD                G001          Q 2024-07-01 520712438379
# 260      USA XG_FOB_USD                G001          Q 2024-10-01 518048376275
# 261      USA XG_FOB_USD                G001          Q 2025-01-01 523834888355
# 262      USA XG_FOB_USD                G001          Q 2025-04-01 553002126621
# 263      USA XG_FOB_USD                G001          Q 2025-07-01 542584602051
# 264      USA XG_FOB_USD                G001          Q 2025-10-01 565784230444
```

Get iData data in R

Below are commands to retrieve information or data from iData:

```
# Get iData data
df <- imf_datatools$get_idata_data('IMF.RES:WEO', 'USA.NGDP.A')

# Get multiple iData data
df <- imf_datatools$get_idata_data('IMF.RES:WEO', 'USA+AUS.NGDP+PPPPC.A')

# Get IMTS (formerly DOT) data
df <- imf_datatools$get_idata_data('IMF.STA:IMTS', 'USA+AUS.XG_FOB_USD.G001.Q')

# Get Bloomberg data
df <- imf_datatools$get_idata_data('IMF.CSF:BBGDL', 'VIX_INDEX.PX_LAST.D')

# Get all databases
dbnames <- imf_datatools$idata_utilities$get_databases()

# Get metadata for series
metadata <- imf_datatools$idata_utilities$get_idata_metadata('IMF.RES:WEO', '.NGDP.')
```

```
# Get WEO country codes and groups
countrydata <- imf_datatools$$get_wEO_country_info()
```

Read and write DMXe data in R

Below are commands to retrieve information or data from DMXe files:

```
# Get single series
dmxefilename <- "//ecnswn12p/ems_shared/pub/datatools/samples/gab.dmx"
df <- imf_datatools$dmxe_utilities$get_dmx_data(dmxefilename, '646BCA')

# Get metadata
df <- imf_datatools$dmxe_utilities$get_dmx_metadata(dmxefilename,
  c('646BCA', '646BM', '646BG'))

# Get all series
serieslist <- imf_datatools$dmxe_utilities$get_all_series(dmxefilename)

# Get all series containing substr
serieslist <- imf_datatools$dmxe_utilities$get_all_series(dmxefilename,
  substr<-'BCA')

# Write out data to DMXe
series <- 'GDP@USECON'
df <- imf_datatools$get_haver_data(series)
# Need to manipulate so rownames is moved to a column.
df$dates <- rownames(df)
rownames(df) <- NULL

filename = 'out.dmx'
dmxe_writer <- import("imf_datatools.dmx_writer_utilities")
dmxe_writer$save_dmx_data(filename, df, datecol='dates')

# Write out metadata to DMXe
metadata <- list("Notes" = "Downloaded from Haver")
dmxe_writer$save_dmx_metadata(filename, series, metadata)
```

Get Haver data in R

Below are commands to retrieve information or data from economic Haver: ¹

```
# Get single series
df <- imf_datatools$get_haver_data('GDP@USECON')

# Get metadata
metadata <- imf_datatools$haver_utilities$get_haver_metadata('GDP@USECON')

# Get databases
metadata <- imf_datatools$haver_utilities$get_databases()

# Get all series in a database
metadata <- imf_datatools$haver_utilities$get_all_series('USECON')
```

¹There is a R library for Haver called *Haver* (documentation at <http://dm-edms.imf.org/cyberdocs/viewdocument.asp?doc=6734588&lib=DMSDR1S>). Users can use this directly instead of calling through Python, although available functions, formatting of data frames, etc. may be different.

```
# Get all metadata in a database
metadata <- imf_datatools$haver_utilities$get_all_metadata('USECON')
```

Get World Bank data in R

Below are commands to retrieve information or data from the World Bank API:

```
2
# Get data
df <- imf_datatools$get_worldbank_data('NY.GDP.MKTP.CD', 'CHN')

# Get all countries
countrydata <- imf_datatools$worldbank_utilities$get_worldbank_countries()

# Get available series
metadata <- imf_datatools$worldbank_utilities$get_all_worldbank_series()

# Get info on a database
info <- imf_datatools$worldbank_utilities$get_database_info()

# Get metadata
metadata <- imf_datatools$worldbank_utilities$get_worldbank_metadata('NY.GDP.MKTP.CD')
```

Combine multiple sources using python script in R

To combine data from multiple sources, users need to copy `imf_datatools/scripts/datacollector.py` and `input.xlsx` from the `datatools` folder to current folder then read in series info from `input.xlsx` and write out to `outname.xlsx`

```
datacollector <- import("datacollector")
# Call the main function in datacollecto
datacollector$main(c("input.xlsx", "outname.xlsx"))
```

5.2 Stata

Starting with Stata 16, released in late 2019, Stata has Python built into it³. This allows Stata to rely on Python to do tasks like data collection that Stata cannot do. Stata offers Python packages that allow integration of data and variables retrieved in Python to be transferred to Stata⁴, and there is a set of Stata `.ado` and Stata help files developed by the Datatoolsteam to allow the usage of the `imf_datatools`.

In R the syntax of running commands is similar to those of Python. However, Stata requires a different syntax for issuing `imf_datatools` commands, so we provide wrapper Stata commands to easily access the data.

Below the output of the `help` output for each command is shown. Running the commands like

```
help idatause
```

will produce clickable instructions.

The functions to view the country information from the EBV and RES WEO Excel file (see Sec. 3.3) are included under `idatause`.

²There is a R library for the World Bank API called `wbstats` (documentation at https://cran.r-project.org/web/packages/wbstats/vignettes/Using_the_wbstats_package.html). Users can use this directly instead of calling through Python, although available functions, formatting of data frames, etc. may be different.

³<https://www.stata.com/new-in-stata/python-integration/>

⁴<https://www.stata.com/python/api16/index.html>

5.2.1 idatause

help idatause

Syntax

```
idatause databases, [keyword()] [searchmode()] [clear |
frame(frame_name[,replace])]

idatause datastructure, database(database_name) [keyword()]
[searchmode()] [clear | frame(frame_name[,replace])]

idatause dimensionid, database(database_name) dimension(dimension_id)
[keyword()] [searchmode()] [clear | frame(frame_name[,replace])]

idatause data, database(database_name) query() [tin(start_time,
end_time)] [panel(dimension_id)] [clear | frame(frame_name[,replace])]

idatause data, database(database_name) query() [tin(start_time,
end_time)] [panel(dimension_id)] [clear | frame(frame_name[,replace])]

idatause metadata, database(database_name) query() [clear |
frame(frame_name[,replace])]

idatause countrygroups [clear]

idatause weocountryinfo [clear]

idatause ebvcountryinfo [clear]
```

options	Description

Main	
databases	dowloads names of the databases
datastructure	dowloads the data structure of the specified database
dimensionid	dowloads the id/code, and other relevant information of the specified dimension
search	search by key word within the specified dimension
data	downloads data
metadata	downloads additional metadata
Options	
keyword()	keyword to search the database
searchmode()	sets to search mode of multiple keywords using or or and
database()	database name
dimension()	dimension id
query()	customized query for data
tin()	time range
panel()	reshapes data into long format
clear	replaces data in memory
frame(frame_name[,replace])	creates new data frame and save the data

in the frame

Description

`idatause` retrieves data via iData API.

Options +-----+ Main +-----+

`databases` downloads names of the databases that are available via iData API.

`datastructure` downloads the data structure of the specified database.

`dimensionid` downloads the id/code, and other relevant information of the specified dimension.

`search` search by key word within a dimension and downloads the search result.

`data` downloads data as specified.

`metadata` downloads metadata as specified.

`weocountryinfo` downloads information on WEO countries (such as country ISO 2-digit code, ISO 3-digit code, capital city, income level, geographic information, and etc.) and country grouping (such as advanced economies, emerging and developing economies, G7, Euro area, LAC, MENA, and etc.).

`countrygroups` downloads information on country groups (such as G001 for World, etc.) and descriptions (such as Advanced Economies, Emerging Market and Developing Economies, G7, LAC, etc.).

`ebvcountryinfo` downloads country information from Enterprise Business Vocabularies (EBV). EBV are sets of common and official terms used in the Fund to describe our organization, work areas, services, member countries, and more. These terms are organized into controlled vocabularies, in the form of taxonomies or flat lists. For details on EBV, please go to <http://www-intranet.imf.org/fundwide/info/EBV/Pages/Home.aspx> or <http://ebv/>.

+-----+
 -----+ Options +-----+

`keyword()` filters the results by keywords. Keywords are NOT case-sensitive, and multiple keywords can be separated by commas.

`searchmode()` sets the relationship between multiple keywords when doing a search, which is `or` or `and`. The default is `or`. For example, `keyword(STA, RES)` without `searchmode()` will return filtered results containing STA or RES. `keyword(STA, CPI) searchmode(and)` will return filtered results containing both STA and CPI.

`database()` specifies the database. For example, `IMF.STA:CPI`, `IMF.RES:WEO`. This option is required for `idatause datastructure`, `idatause dimensionid` and `idatause data`.

`dimension()` specifies the dimension of the database. Dimension id is case-sensitive. This option is required for `idatause dimensionid`.

`query()` allows customized query for data. Data can be retrieved based on the value they take for each dimension. Dimensions must respect the order specified in the data structure and should be separated by a dot "." character. If all value requested, a dimension can be left empty. Multiple values for a dimension are separated by a plus "+" sign.

`tin(start_time, end_time)` allows for time range selection. `start_time` = minimum date or blank to indicate no minimum date; `end_time` = maximum date or blank to indicate no maximum date. `start_time` and `end_time` are included. The default is the whole series.

`tin()` accepts the following date format:

- yearly date: 2020;
- quarterly date: 2020q2 or 2020Q2;
- monthly date: 2010-10 or 2010m10;
- date: 2025-05-26.

Note: `tin(2020, 2021-05)` will return data up to 2021-05, but `tin(2020, 2021-05-15)` will only return data up to 2021-04 when querying monthly data and not 2021-05.

`panel()` reshapes the imported data into long format. Only one dimension id can be specified in `panel()` and is case-sensitive. The default option is to import the data in the wide format.

`clear` replaces data in memory.

`frame(frameName[, replace])` creates new data frame and saves the downloaded data in the frame. If there is no frame named `frameName` in memory, a new frame named `frameName` is created. If `frameName` does exist in memory, you can replace the frame. `replace` clears `frameName` before the data are read to it.

Examples

Get list of databases.

```
. idatause databases, clear
(click to run)
```

Get list of databases and save to data frame named `databases`.

```
. idatause databases, frame(databases, replace)
(click to run)
```

Filter the databases and only include those that contains `sta` or `res`.

```
. idatause databases, keyword(sta, res) clear
(click to run)
```

Filter the databases and only include those that contains `sta` and `cpi`.

```
. idatause databases, keyword(sta, cpi) searchmode(and) clear
```

(click to run)

Get country groups.

```
. idatause countrygroups, clear
```

(click to run)

Get WEO country list.

```
. idatause weocountryinfo, clear
```

(click to run)

Get EBV country info.

```
. idatause ebvcountryinfo, clear
```

(click to run)

```
      +-----+
-----+ CPI +-----
```

1. Get data structure.

```
. idatause datastructure, database(IMF.STA:CPI) clear
```

(click to run)

Filter the data structure by keywords.

```
. idatause datastructure, database(IMF.STA:CPI) keyword(type, freq) clear
```

(click to run)

2. Get dimension id and description.

Dimension id is case-sensitive.

```
. idatause dimensionid, database(IMF.STA:CPI) dimension(COUNTRY)
```

(click to run)

Filter dimension TYPE_OF_TRANSFORMATION by keyword WGT.

```
. idatause dimensionid, database(IMF.STA:CPI) dim(TYPE_OF_TRANSFORMATION)
  keyword(WGT) clear
```

(click to run)

Filter dimension COUNTRY by keywords republic and united.

```
. idatause dimensionid, database(IMF.STA:CPI) dim(COUNTRY)
  keyword(republic, united) searchmode(and) clear
```

(click to run)

3. Get data.

```
. idatause data, database(IMF.STA:CPI) query(USA+CAN.CPI.CP01..A) clear
```

(click to run)

Set time range.

```
. idatause data, database(IMF.STA:CPI) query(USA+CAN.CPI.CP01..M)
  tin(2023Q1, 2024Q2) clear
```

(click to run)

```
. idatause data, database(IMF.STA:CPI) query(USA+CAN.CPI.CP01..M)
  tin(2020, 2021-05-15) clear
```

(click to run)

Set the data in panel/long format.

```
. idatause data, database(IMF.STA:CPI) query(USA+CAN.CPI._T..M)
    panel(COUNTRY) clear
(click to run)
. idatause data, database(IMF.STA:CPI) query(USA+CAN.CPI._T..M) tin(2020,
    2024) panel(INDEX_TYPE) clear
(click to run)
```

3. Get metadata.

```
. idatause metadata, database(IMF.STA:CPI) query(.CPI._T.IX.) clear
```

```
+-----+
----+ WEO Published +-----
```

1. Get data structure.

```
. idatause datastructure, database(IMF.RES:WEO) clear
(click to run)
```

Filter the data structure by keywords.

```
. idatause datastructure, database(IMF.RES:WEO) keyword(type, freq) clear
(click to run)
```

2. Get dimension id (case-sensitive) and description.

```
. idatause dimensionid, database(IMF.RES:WEO) dimension(COUNTRY)
(click to run)
. idatause dimensionid, database(IMF.RES:WEO) dim(INDICATOR) clear
(click to run)
```

3. Get data.

```
. idatause data, database(IMF.RES:WEO) query(USA+AUS.NGDPPC+PCPI.) clear
(click to run)
. idatause data, database(IMF.RES:WEO) query(USA+AUS.NGDPPC+PCPI.)
    panel(COUNTRY) clear
(click to run)
```

3. Get metadata.

```
. idatause metadata, database(IMF.RES:WEO) query(.NGDPR+LUR.) clear
```

Author

Li Tang (ltang@imf.org), Econometric Modelling and Support Team, ITDDE
 Kei Moriya (kmoriya@imf.org), ITDDE
 Minju Jeong (mjeong@imf.org), STADO

5.2.2 dmxeuse

The information below can also be obtained by typing in stata:

```
help dmxeuse
```

Syntax

```
dmxeuse metadata, dmxpath(path) [indicator(indicator)| substr(key words)] [clear |
frame(framename[,replace])]
```

```
dmxuse seriesnames, dmxepath(path) substr(key words) [clear | frame(framename[,replace])]
```

```
dmxuse data, dmxepath(path) indicator(indicator) [scale(unit) freq(frequency) tin(date1,date2)]
[clear | frame(framename[,replace])]
```

options	Description

Main	
metadata	dowloands metadata
seriesnames	dowloands series names
data	downloads data
Data Specification	
dmxepath()	path to DMXe file
indicator()	indicator
substr()	key word to search for series
scale(unit)	scales data by specifield units
freq()	frequency of data
tin()	time range
clear	replaces data in memory
frame(framename[,replace])	creates new data frame and save the data in the frame

Description

dmxuse imports data from DMXe file.

Options

```
+-----+
----+ Main +-----
```

metadata downloads metadata that give information about the DMXe file.

seriesnames only downloads names of the entire series or series which contain the keywords as specified in option substr().

data downloads data as specified.

```
+-----+
----+ Data Specification +-----
```

dmxepath() specifies the location of the DMXe file. This option is required.

indicator() selects indicators. This option is required for dmxuse data.

substr() allows for searching for sereis by keywords. Multiple keywords can be separated by commas. For example, substr(GDP, TX) will return series containing both GDP and TX.

scale(unit) scales data by specified units. Variable labels contains the units and/or currency for reference. scale(default) scales to the original scale stored in the database. Other options for scale() are U/K/M/B/T. If any of these are specified, the values are divided by 1, 1000, 1,000,000, 1,000,0000, 000, 1,000,0000,000,000. default/U/K/M/B/T are case-insensitive. The default is scale(default).

`freq()` specifies the frequency of the imported data. It can be A for annual, Q for quarterly, or M for monthly, or W for weekly, or D for daily. A/Q/M/W/D are case-insensitive. The default is A.

`tin(date1,date2)` allows for time range selection. `date1` = minimum date or blank to indicate no minimum date; `date2` = maximum date or blank to indicate no maximum date. The default is the whole series. For example, `tin(1980,2010)`, `tin(1980q1,2010q1)`, `tin(1980m1,2010m1)`, `tin(2015,)`

`clear` replaces data in memory.

`frame(frameName[, replace])` creates new data frame and saves the downloaded data in the frame. If there is no frame named `frameName` in memory, a new frame named `frameName` is created. If `frameName` does exist in memory, you can replace the frame. `replace` clears `frameName` before the data are read to it.

Examples

Get metadata of all series in a DMXe file.

```
. dmxeuse metadata, dmxpath(\\was.int.imf.org\ecn\ems_shared\pub\datatools\samples\gab.dmx) clear
```

Get metadata of series using keywords search.

```
. dmxeuse metadata, dmxpath(\\was.int.imf.org\ecn\ems_shared\pub\datatools\samples\gab.dmx)
  substr(BCA) clear
```

Get metadata of specified indicators.

```
. dmxeuse metadata, dmxpath(\\was.int.imf.org\ecn\ems_shared\pub\datatools\samples\gab.dmx)
  i(646BCA_BP6) clear
```

Get all series names.

```
. dmxeuse seriesnames, dmxpath(\\was.int.imf.org\ecn\ems_shared\pub\datatools\samples\gab.dmx)
  clear
```

Get available series names and narrow down with `substr()`.

```
. dmxeuse seriesnames, dmxpath(\\was.int.imf.org\ecn\ems_shared\pub\datatools\samples\gab.dmx)
  substr(BCA) clear
```

Get data of specified indicators.

```
. dmxeuse data, dmxpath(\\was.int.imf.org\ecn\ems_shared\pub\datatools\samples\gab.dmx)
  i(646BCA_BP6, 646BXG) clear
```

Get annual data.

```
. dmxeuse data, dmxpath(\\was.int.imf.org\ecn\ems_shared\pub\datatools\samples\gab.dmx)
  i(646BCA_BP6) freq(A) clear
```

Get data in specified time range.

```
. dmxeuse data, dmxpath(\\was.int.imf.org\ecn\ems_shared\pub\datatools\samples\gab.dmx)
  i(646BCA_BP6) tin(2010,) clear
```

Get data in specified scale unit.

```
. dmxeuse data, dmxpath(\\was.int.imf.org\ecn\ems_shared\pub\datatools\samples\gab.dmx)
  i(646BCA_BP6) scale(B) clear
```

5.2.3 sqluse

The information below can also be obtained by typing in stata:

help sqluse

Syntax

```
sqluse databases, server(server name) [clear | frame(framename[,replace])]

sqluse metadata, server(server name) database(database) [indicator(indicators) |
substr(keywords)] [clear | frame(framename[,replace])]

sqluse seriesnames, server(server name) database(database) [substr(keywords)] [clear |
frame(framename[,replace])]

sqluse data, server(server name) database(database) indicator(indicator) [scale(unit)
freq(frequency) tin(date1,date2)] [clear | frame({it:framena dab:replace})]
```

options	Description

Main	
databases	dowloands names of the databases resid on the server
metadata	dowloands the metadata of the database
metadata	dowloands the series names of the database
data	downloads data
Options	
server()	server where data resides
database()	database within the server
indicator()	indicator series, can be separated by commas
substr()	keyword to search for series
scale(unit)	scales data by specifield units
freq()	frequency of data
tin()	time range
clear	replace data in memory
frame(framename[,replace])	creates new data frame and save the data in the frame

Description

sqluse imports data from internal SQL server.

Options

```
+-----+
----+ Main +-----
```

databases downloads database names

metadata downloads metadata of indicator series or series which contain the keywords as specified in option indicator() or substr().

seriesnames only downloads names of the entire series or series which contain the keywords as specified in option substr().

data downloads data as specified.

```
+-----+
----+ Options +-----
```

`server()` specifies the server where data resides. For example, `PRDDMXSQL`, `PRDCSDSQL`. This option is required for all commands.

`database()` specifies the database within the server. For example, `DMX_WDI`, `CSD_FORECAST`. This option is required for `sqluse` metadata and `sqluse` data.

`indicator()` selects indicator series. Multiple series can be separated by commas. For example, `indicator(111.SP.POP.TOTL,193.SP.POP.TOTL)` will return both series. This option is required for `sqluse` data. `indicator()` or `{op ()}` is required for `sqluse` metadata.

`substr()` allows for searching for series by keywords. Multiple keywords can be separated by commas. For example, `substr(111, TAX)` will return series containing both 111 and TAX. `indicator()` or `substr()` is required for `sqluse` metadata.

`scale(unit)` scales data by specified units. Variable labels contains the units and/or currency for reference. `scale(default)` scales to the original scale stored in the database. Other options for `scale()` are `U/K/M/B/T`. If any of these are specified, the values are divided by 1, 1000, 1,000,000, 1,000,000, 000, 1,000,000,000,000. `default/U/K/M/B/T` are case-insensitive. The default is `scale(default)`.

`freq()` specifies the frequency of the imported data. It can be `A` for annual, `Q` for quarterly, or `M` for monthly. `A/Q/M` are case-insensitive. The default is `A`.

`tin(date1:date2)` allows for time range selection. `date1` = initial date; `date2` = final date. The default is the whole series.

`clear` replaces data in memory.

`frame(frameName[, replace])` creates new data frame and saves the downloaded data in the frame. If there is no frame named `frameName` in memory, a new frame named `frameName` is created. If `frameName` does exist in memory, you can replace the frame. `replace` clears `frameName` before the data are read to it.

Examples

Get available databases.

```
. sqluse databases, server(PRDDMXSQL) clear
```

Get available databases and narrow down with `substr()`.

```
. sqluse databases, server(PRDDMXSQL) substr(DMX, VE) f(sql_databases_substr)
```

Get metadata of all series in a database. (NOT RECOMMENDED, MAY TAKE TIME.)

```
. sqluse metadata, server(PRDDMXSQL) database(DMX_WDI)f(sql_metadata_all)
```

Get metadata of series using keywords search.

```
. sqluse metadata, server(PRDDMXSQL) database(DMX_WDI) substr(111, TAX) f(sql_metadata_substr)
```

Get metadata of indicators.

```
. sqluse metadata, server(PRDDMXSQL) database(DMX_WDI) indicator(111.SP.POP.TOTL, 193.SP.POP.TOTL)
```

```
f(sql_metadata)
```

Get all series names.

```
. sqluse seriesnames, server(PRDDMXSQL) database(DMX_WDI) f(sql_seriesnames)
```

Get available series names and narrow down with substr().

```
. sqluse seriesnames, server(PRDDMXSQL) database(DMX_WDI) substr(111, GDP) f(sql_seriesnames_substr)
```

Get data of specified indicators.

```
. sqluse data, server(PRDDMXSQL) database(DMX_WDI) indicator(111.SP.POP.TOTL, 193.SP.POP.TOTL)
  f(sql_data)
```

Get annual data.

```
. sqluse data, server(PRDDMXSQL) database(DMX_WDI) indicator(111.SP.POP.TOTL, 193.SP.POP.TOTL)
  freq(A) f(sql_data_annual)
```

Get data in specified time range.

```
. sqluse data, server(PRDDMXSQL) database(DMX_WDI) indicator(111.SP.POP.TOTL, 193.SP.POP.TOTL)
  tin(2000,) clear
```

5.2.4 dmxewriter

The information below can also be obtained by typing in stata:

```
help dmxewriter
```

Syntax

```
dmxewriter data, dmxpath(path) [frame(framename) indicator(indicator) timevar(time variable) name(name)
freq(frequency) tin(date1,date2)]
```

```
dmxewriter metadata, dmxpath(path) frame(framename) indicator(indicator)
```

```
dmxewriter values, column(column name)
```

```
dmxewriter rename, dmxpath(path) indicator(indicator) newindicator(new indicator)
```

```
dmxewriter delete, dmxpath(path) indicator(indicator)
```

```
dmxewriter deletelogs, dmxpath(path)
```

options	Description
Main	
data	write data to a DMXe file
metadata	write metadata to a DMXe file
values	extract valid metadata values of a column/field
rename	renames an indicator in a DMXe file
delete	deletes the values of indicators from a DMXe file
deletelogs	deletes logs of a DMXe file
data Specification	
dmxpath()	path to the DMXe file
frame()	specifies the frame which contains the data to write to the DMXe file

<code>indicator()</code>	indicators
<code>timevar()</code>	name of the date or time variable which can be used to identify the data to be a time series or a panel data
<code>name()</code>	name of the columns to be written out in the DMXe file
<code>freq()</code>	frequency of data
<code>tin()</code>	time range
<code>column()</code>	column/field name

rename Specification

<code>newindicator()</code>	new indicator name
-----------------------------	--------------------

Description

`dmxewriter` Write data and metadata from a Stata data frame to a DMXe file. It can also modify a DMXe file.

Options +-----+ Main +-----+

`data` writes data from a Stata data frame to a DMXe file.

`metadata` writes metadata from Stata to a DMXe file.

`values` extracts valid metadata values of a column/field

`rename` renames an indicator in a DMXe file.

`delete` deletes values of indicators from a DMXe file. It doesn't delete the holder of the indicators. `freq()` required.

`deletelogs` deletes logs of a DMXe file.

+-----+
----+ Data Specification +-----+

`dmxepath()` specifies the location of the DMXe file. This option is required.

`frame()` the frame which contains the data to write to the DMXe file. If `frame()` is not specific, the default current frame.

`indicator()` specifies indicators to be written out. Indicators are separated by comma. If `indicator()` is not specified for `dmxewriter data`, the default is all variables in Stata data frame. This option is required for `dmxe ename}` and `dmxewriter delete`.

`timevar()` identifies the date or time variable which can be used to identify the data to be a time series or a panel data. If `timevar()` is not specified, the default is variable dates.

`name()` specifies the name of the columns to be written out in the DMXe file. If `name()` is not specified, the default is the variable name in Stata data frame.

`freq()` specifies the frequency of the data to be written out. It can be A for annual, Q for quarterly, or M for monthly, or W for weekly, or D for daily. A/Q/M/W/D are case-insensitive. If the same indicator is available for different frequencies, the default is annual, i.e., A. For example, NGDP is available for annual and quarterly. If quarterly NGDP is requested instead of annual, `freq(Q)` must be specified. `freq()` is required for `dmxewriter delete`.

`tin(date1,date2)` allows for time range selection. `date1` = minimum date or blank to indicate no minimum date; maximum date or blank to indicate no maximum date. The default is the whole series. For example, `tin(1980q1,2010q1)`, `tin(1980m1,2010m1)`, `tin(2015,)`

`column()` specifies the name of the column/field. This option is required for `dmxewriter` values.

Examples

+-----+

-----+ WEO +-----

1. Get annual data from EcOS.

```
. ecosuse data, database(WEO_WEO_PUBLISHED) country(111, 193, 196) indicator(NGDP, PCPI) f(ecos_A, replace)
(click to run)
```

Write data to a DMXe file.

```
. dmxewriter data , dmxepath(C:/temp/dmxewriter_example.dmx) frame(ecos_A) indicator(_111NGDP_A, _193PCPI_A)
timevar(dates) name(111NGDP.A, 193PCPI.A)
(click to run)
```

2. Get quarterly data from EcOS.

```
. ecosuse data, database(WEO_WEO_PUBLISHED) country(111) indicator(NGDP) freq(Q) f(ecos_Q, replace)
(click to run)
```

Write data to a DMXe file.

```
. dmxewriter data, dmxepath(C:/temp/dmxewriter_example.dmx) frame(ecos_Q) indicator(_111NGDP_Q) timevar(dates)
name(111NGDP.Q)
(click to run)
```

3. Get monthly data from EcOS.

```
. ecosuse data, database(ECDATA_IFS) country(111) indicator(ENEER_IX) freq(M) frame(ecos_M, replace)
(click to run)
```

Write data to a DMXe file.

```
. dmxewriter data, dmxepath(C:/temp/dmxewriter_example.dmx) frame(ecos_M) indicator(_111ENEER_IX_M) timevar(dates)
name(111ENEER_IX.M)
(click to run)
```

4. Get daily data from EcOS.

```
. ecosuse bloomberg, ticker(VIX Index) field(PX_LAST) frame(ecos_D, replace)
(click to run)
```

Write data to a DMXe file.

```
. dmxewriter data, dmxepath(C:/temp/dmxewriter_example.dmx) frame(ecos_D) indicator(VIX_IndexPX_LAST_D)
timevar(dates) name(VIX_IndexPX_LAST.D)
(click to run)
```

5. Specify option `tin()` to restrict the time range of the data to be written out.

```
. dmxewriter data, dmxepath(C:/temp/dmxewriter_example.dmx) frame(ecos_A) indicator(_111PCPI_A) timevar(dates)
name(111PCPI.A) tin(1990, 2020)
(click to run)
```

+-----+

-----+ Haver +-----

1. Get annual data from Haver.

```
. frame create haver_A
```

(click to run)

```
. frame change haver_A
(click to run)
. import haver gdpa@usecon, clear
(click to run)
Write data to a DMXe file.
. dmxewriter data, dmxpath(C:/temp/dmxewriter_example.dmx) frame(haver_A) indicator(gdpa_usecon) timevar(ti
    name(GDPA@USECON)
(click to run)

2. Get daily data from Haver.
. frame create haver_D
(click to run)
. frame change haver_D
(click to run)
. import haver ffed@daily, clear
(click to run)
Write data to a DMXe file.
. dmxewriter data, dmxpath(C:/temp/dmxewriter_example.dmx) frame(haver_D) indicator(ffed_daily) timevar(tim
    name(FFED@DAILY)
(click to run)

+-----+
----+ Modify metadata +-----+

Write metadata to a DMXe file.
1. Modify column Descriptor. Descriptor can take any string value.
. dmxewriter metadata, dmxpath(C:/temp/dmxewriter_example.dmx) indicator(111NGDP.A) column(Descriptor) valu
    domestic product, current prices")
(click to run)

2. Modify column Scale. Scale can take only pre-specified values.
. dmxewriter metadata, dmxpath(C:/temp/dmxewriter_example.dmx) indicator(111NGDP.A) column(Scale)
    value("Billions")
(click to run)

3. Previous example shows that Scale can take only pre-specified values. dmxewriter values can extract value
    values for a field.
. dmxewriter values, column(Scale)
(click to run)

Please note that not all columns can be modified. If a column is not a valid writable DMXe field, the action
    ignored, and Stata will report a warning.

+-----+
----+ Modify a DMXe file +-----+

1. Rename an indicator in a DMXe file.
. dmxewriter rename, dmxpath(C:/temp/dmxewriter_example.dmx) indicator(FFED@DAILY) newindicator(FFED)
(click to run)

2. Delete values of indicators from a DMXe file.
. dmxewriter delete, dmxpath(C:/temp/dmxewriter_example.dmx) indicator(FFED, VIX_IndexPX_LAST.D) freq(D)
(click to run)
```

3. Delete log file of a DMXe file.

```
. dmxewriter deletelogs, dmxepath(C:/temp/dmxewriter_example.dmx)
```

(click to run)

Chapter 6

Summary

This document provided an overview of the `imf_datatools` Python library. Topics such as installation, available resources and functions were covered, as well as usage through R and Stata. At this time there is no support for Matlab or EViews. For Matlab users, it is recommended that Python and the `imf_datatools` library be used as a system command to save any data in csv format and then reading that csv file in using Matlab. This way the `imf_datatools` can be used seamlessly from within Matlab.

Further planned enhancements include

- better handling of exceptions so they can be raised in Stata and R
- better handling of frequency of series
- distinguish between data not available and connection failures
- enhancements to the datacollector scripts and further output

The latest info on the project is available at the datatools website: <http://datatools>, and a link to the latest version of this document is also available.

6.1 Acknowledgements

The `imf_datatools` were developed while providing support to users as part of the Econometric Support team, starting around July 2018. Many thanks are due to everybody who provided support and feedback. The first step in adding true value was connecting to EcOS, many thanks to Jerry Chaves, Jobin Maliakal and Rajesh Nilawar and the EcOS/DMX team for their technical support. The possible outreach of the tools increased significantly with the introduction of Python integration in Stata 16, many thanks to Li Tang (Econometric Support) for her support, wisdom and knowledge of what Fund users want. Adeleke Adeyemi (Econometric Support) was kind enough to write the R part of this documentation and provide support for R users. Max Yarmolinski was kind enough to take on much of the underappreciated work of documentation and testing. Thanks to my economist friends Romain Lafarguette and Futoshi Narita for much-appreciated feedback and encouragement, and to the Automation Project team members who provided support as part of an iLab-sponsored project.

Many thanks to the following users for their interest and providing feedback:

- Ryan Yost
- Antoine Arnoud
- Chengyu Huang
- Di Yang
- Shiyao Wang